LoopFuzz: Integrated Closed-Loop Control for LLM-Guided Stateful Fuzzing of Response-Visible Protocols

Ketang Chen[a], Xiaolei Ren[a,*]

[a]*Macau University of Science and Technology, Avenida Wai Long, Taipa, Macau, China*

Abstract

Stateful protocol fuzzing is difficult because syntactically valid messages generated by large language models (LLMs) can still violate the live session state. We present LoopFuzz, an integrated closed-loop controller for LLM-guided fuzzing of textual or semi-textual request-response protocols with response-visible progress. LoopFuzz treats model outputs as bounded proposals: request candidates must pass parseability, response acceptability, state-reachability, and gain checks before queue promotion, while scheduling actions remain bounded hints. On nine AFLNet/ChatAFL-lineage targets, repeated-run archives show corrected improvements in response-derived inferred protocol state machine (IPSM) edges on eight targets versus AFLNet and six versus the controlled ChatAFL baseline; branch-coverage improvements remain corrected-significant on seven and four targets, respectively. Forked-daapd is a design-boundary case: LoopFuzz grows slightly more response-derived state structure but obtains lower and more variable branch coverage, showing that IPSM edges are not a universal coverage proxy. Preliminary triage confirms replayable crash or protocol-oracle artifacts but does not support vulnerability-yield ranking. Auxiliary NSFuzz, Mosquitto, and MBFuzzer rows are diagnostics, not same-pipeline rankings. Because the archive lacks a *LoopFuzz-no-admission* arm and candidate-level admission logs, the results support the integrated control policy, not an isolated causal estimate of the full admission formula.

Keywords: Protocol fuzzing, Large language models, Closed-loop control, Stateful testing, Counterexample-guided local repair

1. Introduction

Network protocol implementations remain difficult security targets because correctness depends on interaction history rather than isolated inputs [1–5]. A File Transfer Protocol (FTP) server, for example, rejects a well-formed `PASS` command if no valid `USER` command has established the login state. Unlike file fuzzing, protocol fuzzing must preserve message syntax, temporal order, and request-response progress across a multi-step session [5–10].

Traditional coverage-guided fuzzers work well for stateless parsers. They fit network servers poorly when later behavior depends on earlier valid exchanges [11–15]. Stateful greybox fuzzers such as AFLNet and StateAFL partially close this gap. They infer progress from responses and replay valid traces [1–4, 7, 16, 17]. Even so, byte-level mutation still struggles to synthesize the context-sensitive messages needed to cross deep protocol boundaries.

Large language models (LLMs) offer a plausible source of such messages. ChatAFL and later LLM-assisted fuzzers show that a model can extract structure from specifications, logs, or documentation and produce fluent candidates [18–25]. But fluency is not enough. A request can be grammatically plausible and still be invalid for the live session state. Such executions spend budget without improving queue quality.

LoopFuzz places a runtime admission controller between LLM generation and queue admission. It treats the LLM as a hypothesis generator rather than an oracle: hypotheses are derived from Request for Comments (RFC) fragments, seed traces, and observed responses; failed executions become counterexamples for local repair; and inferred protocol state machine (IPSM) feedback steers frontier scheduling and plateau-triggered assistance. The name *LoopFuzz* emphasizes closed-loop admission and local repair, not formal verification of protocol semantics, model behavior, or server correctness.

The scope is deliberately narrower than universal protocol fuzzing. LoopFuzz is designed and evaluated for textual or semi-textual request-response protocols where candidate parseability can be checked locally and response-derived progress is visible to the fuzzer. Binary, encrypted, or tightly formatted protocols such as Transport Layer Security (TLS), Datagram Transport Layer Security (DTLS), QUIC, Secure Shell (SSH), Internet Protocol Security (IPsec), and similar targets would require dedicated adapters for message construction, rejection classification, and state extraction before the same control policy could be evaluated. We do not claim that the current evidence validates LoopFuzz on those protocol families.

The methodological contributions of this paper are as follows:

- **Hypothesis-Driven Modeling.** LoopFuzz derives message grammars and field constraints from RFC fragments, seed traces, and observed responses, and maintains them

*Corresponding author: Xiaolei Ren. Email address: xlren@must.edu.mo

Email addresses: 3250006913@student.must.edu.mo (Ketang Chen), xlren@must.edu.mo (Xiaolei Ren)

as explicit, auditable runtime hypotheses.

- **Runtime-Evidence Boundary.** LoopFuzz inserts a runtime-evidence boundary between the LLM and the durable queue. It checks structure before trial execution and uses bounded execution evidence to evaluate response acceptability, state reachability, and downstream gain before durable queue promotion.
- **Counterexample-Guided Local Repair.** LoopFuzz preserves failed request-response traces as counterexamples and exposes a prompt-constrained, audit-logged local repair path; in this evaluation, we treat repair as an auditable control mechanism rather than an independently quantified source of endpoint gains or a hard single-patch guarantee.
- **State-Aware Scheduling.** LoopFuzz leverages IPSM topology, state productivity, and adaptive plateau detection to prioritize underexplored frontier states and to request model assistance only when mutation alone stalls.

Accordingly, the paper evaluates LoopFuzz as an integrated campaign-control policy for response-visible request-response protocols, not as a standalone generator, standalone admission formula, or universal protocol adapter. The evidence should be read as an end-to-end control result within this response-derived setting.

Evaluation summary. In the primary lineage evaluation, the repeated-run archive shows improved response-derived state exploration and branch coverage on most AFLNet/ChatAFL targets. External state-aware artifacts are reported only as diagnostics, and the archive does not isolate the independent causal effect of the complete $P/U/R/G$ admission boundary.

2. Background and Motivation

Table 1 contrasts LoopFuzz with representative fuzzers. ChatAFL-style systems use protocol and interaction context to guide LLM generation. LoopFuzz studies a stricter campaign-control setting in which generated proposals are mediated by runtime evidence before they influence a long-running stateful campaign. The evaluation measures this setting as an integrated strategy rather than as an isolated test of the admission boundary alone.

2.1. Coverage-Guided Fuzzing vs. Network Targets

A coverage-guided fuzzer operates through a repeated feedback cycle of seed selection, mutation, execution, coverage measurement, and novelty-based queue update [12–15]. This loop is effective for file parsers, but network targets impose three constraints that matter here:

1. **Temporal state dependence:** server behavior depends on the order and semantics of earlier messages, not only on the current payload [1–4].
2. **Strict message format:** protocols require delimiters, field layouts, and often checksum or length consistency, so many mutated messages die before they reach deeper logic [4, 26, 27].
3. **Execution instability:** sockets, forking daemons, and asynchronous response handling complicate both coverage collection and campaign throughput [17].

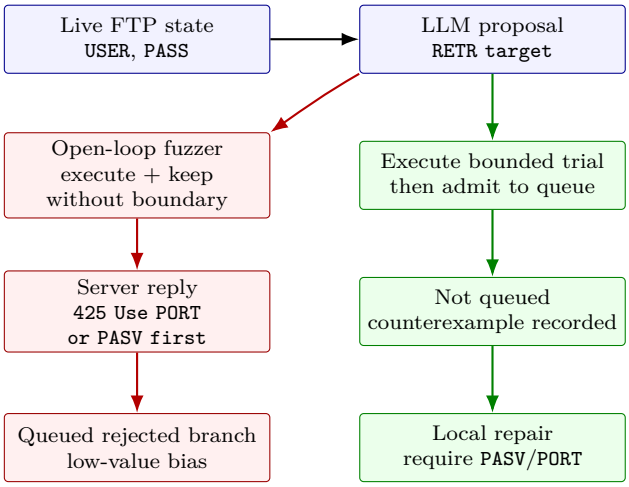


Figure 1. Illustrative, non-archived FTP semantic-drift pattern with bounded post-trial admission and counterexample evidence

2.2. Stateful Greybox Fuzzing (SGF)

SGF addresses this gap by treating the target through an IPSM. In the AFLNet lineage, this machine is an approximate response-derived graph, not a deterministic model of the protocol semantics. We use the lightweight abstraction $\mathcal{M} = (Q, \widehat{\Delta}, q_0)$ to describe the scheduling signal, where:

- Q denotes the set of response-derived state identifiers;
- $\widehat{\Delta} \subseteq Q \times Q$ denotes the observed transition relation induced by concrete executions; and
- q_0 denotes the initial connection state.

AFLNet-style fuzzers infer Q and $\widehat{\Delta}$ from observed responses and use that structure to bias later exploration. Response codes, response texts, and adapter summaries create state identifiers and error hints; they are not modeled as a separate output function here. The abstraction can be partial and history-dependent because asynchronous replies, races, timeouts, daemon state, and environment can make the same sequence expose different response evidence. LoopFuzz therefore uses the IPSM as a scheduling and progress signal, not as a complete protocol semantics model.

2.3. LLM Integration into Protocol Fuzzing

LLMs make it easier to propose structure-aware protocol messages from RFC text, interaction history, or observed responses. That helps with syntax and message diversity, but it does not solve the harder control problem: a fluent request can still be wrong for the live session state.

2.4. A Motivating Example: Semantic Drift

Consider the FTP session in Fig. 1. After `USER` and `PASS`, the LLM proposes `RETR target`; the request is syntactically fluent but premature because no `PASV` or `PORT` data channel exists. We call this failure *semantic drift*: a locally plausible request violates the live session precondition. LoopFuzz handles it through runtime validation. The controller checks local structure, runs a bounded trial, treats the rejection response as negative evidence, avoids durable queue promotion, and can record the failed trace as counterexample evidence for a local `PASV/PORT` repair.

This example is illustrative rather than empirical trace evidence. The selected archives preserve run-level `fuzzer_stats`,

Table 1. Representative fuzzer taxonomy

System	Input prior	State signal	Admission rule	Online repair
Coverage-guided mutation fuzzing [11, 12]	Byte mutation	None	None	No
AFLNet [1]	Packet capture (PCAP) mutation	Response	Novelty	No
StateAFL [2]	PCAP mutation	Memory	Novelty	No
ChatAFL [18]	Context-guided LLM generation	Interaction history	No runtime admission boundary	No
LoopFuzz (ours)	**LLM hypotheses**	**IPSM frontier + validation**	**Runtime admission**	**Yes**

`plot_data`, queue/IPSM artifacts, response-derived files, and raw plateau replies, but not normalized event rows joining state summary, LLM candidate, bounded-trial response, admission-predicate decision, promotion flag, and repair consequence. Semantic drift is therefore a design motivation; its prevalence remains a candidate-level logging gap.

3. Related Work

We position LoopFuzz against the prior lines that are closest in mechanism and evaluation.

Earlier structure-aware fuzzing used explicit structure, inferred snippets, token constraints, or grammar guidance to preserve validity [26–32]. These systems improve input quality, but they do not address the control problem that arises when an external model proposes state-dependent protocol messages.

Stateful greybox and state-aware network-service fuzzing address protocol interaction more directly. AFLNet, StateAFL, NSFuzz, and the broader SGF line infer or expose state structure from server responses, process memory, or program variables [1–3, 16, 33]. They then use that structure to prioritize rare or productive traces. These systems define the broader comparison landscape, but they do not all expose the same state abstraction, instrumentation path, or archived endpoint metrics.

Later systems extend this idea to packet-sequence exploration, format-sensitive protocol fuzzing, and communication-heavy environments [4–8, 34–38]. LoopFuzz asks a narrower question: given an AFLNet-style response-derived IPSM and the ChatAFL-style LLM proposal path, does an integrated runtime-admission strategy make the campaign more reliable? AFLNet and ChatAFL are therefore the closest implementation-lineage baselines. StateAFL, SGF, NSFuzz, Bleem, Logos, and MBFuzzer remain important external comparison points; their different state abstractions, instrumentation paths, and artifact semantics motivate the separate diagnostic tables in the evaluation.

Adjacent work studies state-aware or interaction-heavy fuzzing in robotic policies, firmware, Linux drivers, middleware, hardware, and embedded systems [39–45]. Although not all are protocol fuzzers, they reinforce the point that state feedback can dominate raw mutation quality.

Target-aware scheduling and semantic validation form another relevant line. Directed greybox fuzzing, seed selection, and strategy adaptation study how to steer budget toward high-value behaviors [11–15, 46, 47]. Semantic validation and fuzz-driver generation show that stronger inputs often require an explicit acceptance interface [48, 49]. LoopFuzz applies that lesson to LLM proposals.

Our repair loop also borrows the broad counterexample-driven intuition from counterexample-guided abstraction refinement (CEGAR) in formal verification [50]. The analogy is limited: LoopFuzz records failed executions as concrete evidence for local message-hypothesis patches, but it does not construct a formal abstraction, prove refinement soundness or completeness, or compute minimal counterexamples.

Work on dependency inference, data-flow guidance, and learned signals outside protocol fuzzing also supports treating guidance as bounded evidence rather than as an oracle [51–53].

LLM guidance addresses the synthesis bottleneck more directly. ChatAFL performs context-guided LLM generation from RFCs and interaction histories [18]; adjacent systems use LLMs for general input generation, documentation-guided constraints, device fuzzing, or fuzz-driver generation [19–23, 54]. Our work is closest to ChatAFL, and differs by adding an explicit runtime admission boundary: LoopFuzz asks whether a proposal should be admitted, whether its hypothesis survives runtime evidence, and how the IPSM should guide the next query.

Benchmarking practice is also part of the context. ProFuzzBench established a reusable baseline for repeated protocol-fuzzing experiments [16]. Recent methodology work shows how easily comparisons become unstable without fixed budgets, repeated trials, and careful interpretation of coverage signals [55–57]. We follow that guidance because runtime-validated assistance is especially sensitive to campaign history.

4. LoopFuzz Design

Fig. 2 summarizes the three coupled loops in LoopFuzz: the Grammar Hypothesis Loop, the Fuzz Execution Loop, and the LLM Intervention Loop.

LoopFuzz preserves the stateful greybox fuzzing execution path while placing a control boundary between LLM output and durable queue admission. Request-producing outputs change the queue only after structural checks, bounded execution, and post-trial evidence support promotion. Non-request actions, such as target-state redirection or seed prioritization, remain schema-checked scheduling hints whose value is mediated by later AFLNet-style novelty logic. The four coupled components are grammar hypotheses, runtime admission, counterexample-guided repair, and state-aware scheduling.

Figure 2 separates the method into three analytical loops. The Grammar Hypothesis Loop builds and repairs message hypotheses from RFC knowledge, seed traces, and observed responses. The Fuzz Execution Loop performs state selection, mutation, bounded execution, IPSM update, and novelty checking. The LLM Intervention Loop is activated by a periodic plateau monitor, not by every execution, and returns either a validated continuation or a bounded control action.

The labels in Fig. 2 are control interfaces rather than independent empirical claims. Tier-1 sampling accumulates evidence, Tier-2 repair uses counterexamples, schema validation checks replies, and bounded action dispatch maps a validated request proposal into a trial. Only request candidates that pass post-trial admission become durable queue entries; scheduling-only actions remain weaker hints.

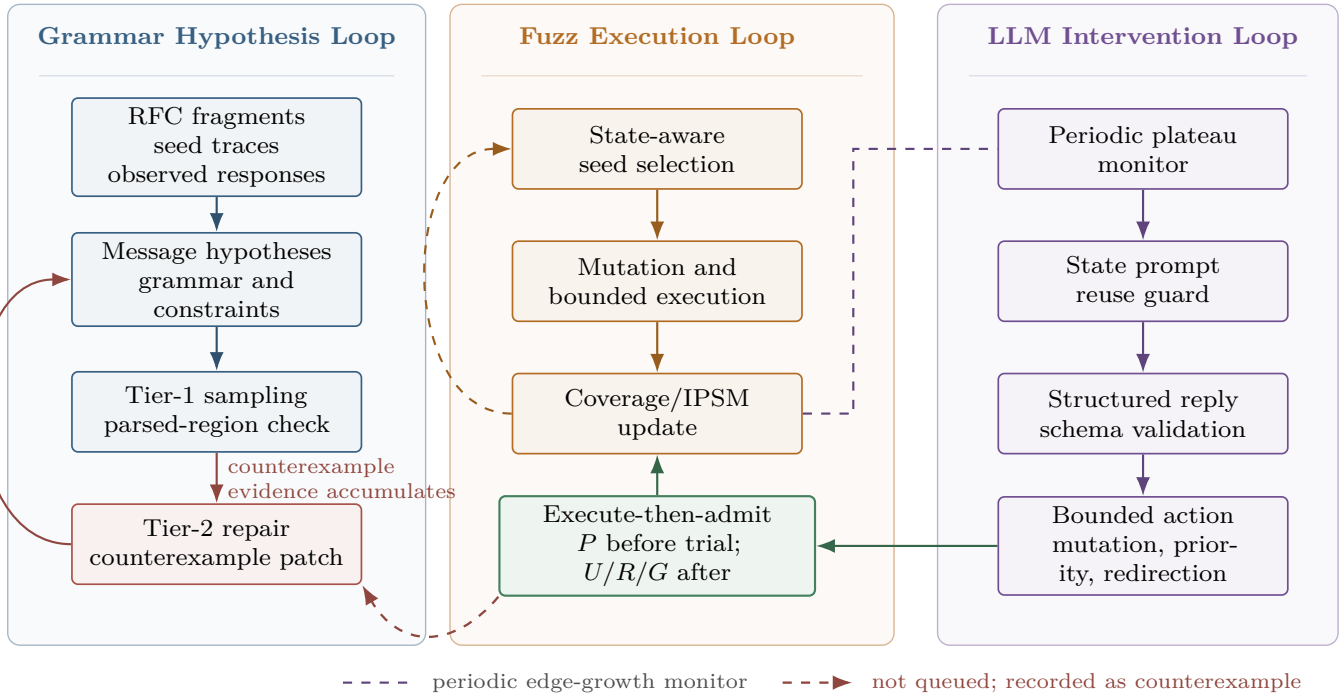


Figure 2. LoopFuzz closed-loop architecture with bounded policy proposals, plateau-gated repair, and execute-then-admit request validation

4.1. Grammar Hypothesis Loop

Let $\mathcal{L}(c, s, \Sigma)$ denote the LLM generation function under prompt context c, state summary s, and protocol knowledge Σ. LoopFuzz uses $\mathcal{L}$ in two stages: bounded initial enrichment preserves ChatAFL's template and seed path, while a lazily initialized hypothesis layer is activated at the first plateau event and then updated through sampled validation and local repair.

The Grammar Hypothesis Loop draws on three evidence sources when they are available: RFC fragments, queued seed traces or packet examples, and observed server responses, including response codes and error texts. For each message type m, the loop maintains a hypothesis $h_m = (G_m, \Phi_m)$. Here, G_m is an Augmented Backus–Naur Form (ABNF)-style or template-style production, and Φ_m stores field constraints such as length bounds, enumerations, token dependencies, and simple cross-field relations. This representation is stricter than free-form text, lighter than a full specification, and cheap to validate online.

The evidence sources need not be complete. LoopFuzz requires testable hypotheses rather than a full formal protocol specification; runtime admission decides whether those hypotheses can affect the campaign.

4.2. Runtime Admission Controller

The runtime admission controller makes queue-admission decisions for request-producing model outputs in two stages. Stage 1 checks structural well-formedness and local parseability before a proposal is sent. Stage 2 runs a bounded trial and evaluates the observed response, inferred state movement, and coverage or IPSM gain. A request candidate x is retained only if

$$A(x) = P(x) \wedge U(x) \wedge R(x) \wedge G(x),$$

where P denotes pre-trial parseability and U, R, and G are post-trial predicates. U captures response-grounded acceptability; R requires a usable response-derived state sequence, target/frontier reachability, or escape from a rejection sink; and G asks whether the trial adds coverage, unseen IPSM structure, or queue value relative to the pre-trial campaign state. If P fails, the proposal is not executed. If any post-trial predicate fails, the trace may update counterexample or productivity evidence, but it is not promoted. Non-request control actions are outside this formula: they undergo schema and identifier checks, alter only scheduling metadata, and are evaluated indirectly by later executions.

Table 2 gives the operational definition and the minimum event fields needed to replay an admission decision. Because the current archive lacks normalized admission-event logs, the table is a controller specification and required log schema, not a recovered mechanism-results table.

The distinction between R and G is intentional. R asks a local trial question: did the candidate produce a usable state sequence or reach the scheduled target/frontier state? G asks a global campaign question: did this trial add something not already present in the campaign's coverage map, IPSM, or favored queue? A trial can therefore pass U and R yet fail G. For example, a candidate may reach the scheduled target state and receive a non-rejection response, so R passes; if the same response-derived transition was already registered by another execution and the trial produces no coverage delta or favored queue entry, then G fails and the candidate is not promoted.

The cleanest ablation for this design would be *LoopFuzz-no-admission*: it would keep the same model endpoint, prompts, query budget, plateau triggers, and scheduling policy, but skip the $P/U/R/G$ checks and directly enqueue request-producing model suggestions while retaining the same schema checks for control actions. The current archive does not contain that condition, so the experiments evaluate LoopFuzz as an integrated control policy.

The FTP example in Fig. 1 illustrates the same logic. `RETR target` may pass P, but a live `425` response would fail U and provide no useful R/G evidence, so the candidate would not be queued and would instead become repair evidence.

Table 2. Request-admission predicates and audit fields

Pred.	Input	Pass	Fail	Log/effect
P Parse	Pre: action/request, h_m.	Schema; fields; bounds; delimiters; local parse.	Malformed action; missing field/term.; type/len./parse err.	Id, type, flags, err., hyp.; no exec.
U Accept	Post: resp., code/state, timeout/reset, adapter.	Non-reject state seq.; or $h = 1 \to 2$, $N_s \geq 10$, $p_s \geq 0.1$.	4xx/5xx; timeout/reset/drop; no state seq.; unparsable.	Extractor, code, timeout, h, N, D, p, reason, override; no promote.
R Reach	Post: src, state seq., target/frontier.	Non-reject seq.; target/frontier hit; sink escape.	Same reject sink; no seq.; unmapped state.	Src/dst, seq., target, escape, productivity; novelty deferred.
G Gain	Post: coverage, IPSM novelty, favored status.	New coverage; new IPSM node/path/edge; favored entry.	U/R pass; no coverage/IPSM/queue gain.	Bit/branch/IPSM deltas, favored, gain reason, promote flag; all-pass promotes.

4.2.1. Structured Output and Parseability

The plateau stage accepts either a concrete next-request candidate or a bounded structured action. Runtime validation checks structural well-formedness, field types, length bounds, delimiters, terminators, and action-schema consistency, then tests local parseability against current message hypotheses. This can reject malformed interventions before trial execution; usefulness is decided only by the post-trial predicates in Table 2.

4.2.2. Response-Grounded Acceptability

Parseability alone does not rule out semantic prematurity. LoopFuzz therefore classifies bounded-trial responses and treats a non-rejection response as positive evidence. For the primary text protocols, response-code priors cover the FTP, Simple Mail Transfer Protocol (SMTP), Real Time Streaming Protocol (RTSP), Hypertext Transfer Protocol (HTTP), Session Initiation Protocol (SIP), and Internet Printing Protocol (IPP) adapters: state ids are classified by $\lfloor s/100 \rfloor$, and classes 4 and 5 initialize $h(s) = 1$ as likely rejection. Forked-daapd uses the HTTP-style extractor because Digital Audio Access Protocol/Digital Audio Control Protocol (DAAP/DACP) requests are HTTP/1.1 requests with DAAP paths and query parameters. Timeouts, early disconnects, resets, and responses without a recoverable state sequence are negative execution evidence.

The earlier word "rescue" has a narrow implementation meaning. Protocol adapters map response bytes to state ids; they do not provide target-specific rescue thresholds. The only implemented rescue from a rejection prior is the uniform behavioral override in Table 4: $h = 1$ may become $h = 2$ when $N_s \geq 10$ and $p_s = D_s/(N_s + 1) \geq 0.1$. Conversely, $N_s \geq 30$ and $p_s < 0.005$ demotes initially unknown states for scheduling but is not a positive U rescue. The controller therefore asks a narrow post-trial question: did live target behavior support promoting this candidate?

Tables 3 and 4 are intentionally rule-level. The current archive does not preserve normalized candidate-level fields for extractor id, response code, timeout flag, $h(s)$, N_s, D_s, p_s, override flag, and post-trial novelty outcome. Therefore, the tables make the adapter and uniform-policy boundary reproducible, but they cannot quantify per-target rescue frequency, rejection reasons, or the effect of rescued states on endpoint coverage.

4.2.3. State Reachability and Coverage Gain

The final admission question is whether the trial helped the campaign. LoopFuzz separates reachability R from gain G: a candidate may reach the intended state, yet fail durable promotion if the same IPSM edge is already known and the trial adds no branch delta or favored queue entry. The controller therefore uses IPSM-derived reachability and AFLNet-style novelty signals, not text quality alone. Each bounded trial is still a real target interaction charged to the same 24-hour wall-clock budget, and the archive cannot quantify predicate-failure frequencies, avoided queue pollution, displaced mutation executions, or per-target trial time.

4.3. Counterexample-Guided Local Repair

When sampled validation fails repeatedly, LoopFuzz records a concrete counterexample. The record contains the failure-triggering request fragment, the observed response, and the mismatch against the current hypothesis. Repair is deliberately prompt-constrained and audit-logged. Instead of asking the model to rewrite the full protocol description, the refinement prompt instructs it to make the minimal local change, preferably modifying at most one field constraint, production rule, or schema field, and to report the intended `changed_field`. The implementation then performs a selective merge, logs the changed fields, and warns when the returned hypothesis changes more than two major fields. The resulting update is reintegrated into protocol matching and re-evaluated by later sampled validation.

This repair boundary is weaker than formal grammar repair: the implementation neither rejects every multi-field repair nor performs hard abstract syntax tree (AST) differencing that proves a one-field or one-production patch. For carriage-return/line-feed (CRLF)-delimited text messages, it attempts lightweight line-level counterexample minimization before storing a failed trace; this reduces prompt noise but gives no semantic minimality guarantee. We therefore claim prompt-constrained, audit-logged local repair, not complete protocol inference or hard single-patch enforcement.

4.4. State-Aware Scheduling and Plateau-Triggered Assistance

Reducing malformed generations is not enough: stateful fuzzers can still revisit dense IPSM regions while missing narrow protocol bridges. LoopFuzz therefore treats IPSM feedback as a first-class scheduling signal and augments state selection with a topology-and-productivity bonus $\beta(s)$. The policy separates

Table 3. Protocol-adapter rules

Targets	Opt.	State id	Reject prior
LightFTP, bftpd, ProFTPD, Pure-FTPd	`-P FTP`	Line → 3-digit code; start 0.	4xx/5xx → $h = 1$.
Exim	`-P SMTP`	Line → 3-digit code; start 0.	4xx/5xx → $h = 1$.
Live555	`-P RTSP`	`RTSP/` status → 3-digit code; others ignored.	4xx/5xx → $h = 1$.
Kamailio	`-P SIP`	`SIP/` status → 3-digit code; others ignored.	4xx/5xx → $h = 1$.
Forked-daapd	`-P HTTP`	HTTP status → 3-digit code; DAAP paths not states.	4xx/5xx → $h = 1$.
Lighttpd1	`-P HTTP`	HTTP status → 3-digit code; start 0.	4xx/5xx → $h = 1$.

Table 4. Uniform policies

Policy	Scope	Rule	Effect
Timeout	All targets	`-W/-w`; timeout/reset/drop	No positive U.
Unparsable response	All targets	No post-initial state sequence	No positive U/R.
Rescue	Uniform	$h = 1 \to 2$ iff $N_s \geq 10$, $p_s = D_s/(N_s + 1) \geq 0.1$	No per-target threshold.
Low productivity	Uniform	$h = 0$, $N_s \geq 30$, $p_s < 0.005$	Scheduling demotion only.
G novelty	All targets	New coverage/IPSM/favored entry	Shared gain rule.

low out-degree from useful low out-degree, so response-derived dead ends are not rewarded only because $\deg^+(s)$ is small:

$$\beta(s) = \beta_{\text{frontier}}(s) \cdot \rho(s).$$

Let $\deg^+(s) = |\{s' \in Q : (s, s') \in \widehat{\Delta}\}|$ be the current out-degree of state s in the observed IPSM relation introduced in Section 2.2. The implementation computes this value from the outgoing edges of the Graphviz IPSM node for s; if s has not yet appeared in Q, the frontier rule treats it as an unseen state before evaluating out-degree. Let $n_s = \textit{selected_times}(s)$ and $p_s = \textit{productivity}(s)$. The frontier term and penalty term are:

$$\beta_{\text{frontier}}(s) = \begin{cases} 4.0 & \text{if } s \notin Q \\ 4.0 & \text{if } \deg^+(s) \leq 1 \\ 2.0 & \text{if } \deg^+(s) \leq 3 \\ 1.0 & \text{otherwise} \end{cases},$$

$$\rho(s) = \begin{cases} 0.5 & \text{if } h(s) = 1 \\ 0.7 & \text{if } h(s) = 0,\ n_s \geq 30,\ p_s < 0.005 \\ 1.0 & \text{otherwise} \end{cases}.$$

The productivity term is a cumulative queue-productivity ratio, not a finite sliding-window coverage rate. Let $D_s(t)$ be the cached `paths_discovered` counter for state s at time t, and let $N_s(t)$ be its `selected_times` counter. The implementation increments N_s whenever the scheduler selects s as the target state for a fuzzing round. It increments D_s once when a non-dry-run execution saved as an interesting queue path is processed while s is the current target state. The cached value used by the scheduler is

$$p_s(t) = \frac{D_s(t)}{N_s(t) + 1}.$$

Newly created states initialize $D_s = N_s = 0$ and $p_s = 0$; the $+1$ denominator guards against zero division. There is no finite window W: the effective window is the state lifetime since creation. Thus p_s approximates queued-path yield when targeting s, not per-execution coverage-bit discovery, IPSM-edge discovery, or favored-queue rate. The 0.005 cutoff applies only after $N_s \geq 30$ selections and is reported as an auditable controller constant rather than a tuned optimum. Here $h(s) \in \{0, 1, 2\}$ means unknown, likely rejection, and confirmed productive. The topology bonus is applied first, and the acceptability/productivity penalty reduces investment in rejection-heavy or repeatedly unproductive states without excluding them solely by degree.

The constants above are a reported control policy, not a claim of optimality. Table 5 gives the only implemented online transitions: $1 \to 2$ for productive rejection-coded states and $0 \to 1$ for repeatedly unproductive unknown states. There is no recovery-to-unknown rule, confirmed-productive demotion, finite-window streak counter, or full sensitivity analysis. The 64-call intervention budget separately prevents unbounded LLM overhead.

A complementary graph stagnation check is Message Queuing Telemetry Transport (MQTT)-only. After more than 50 consecutive rounds without IPSM-node growth, the scheduler uses random state selection with 40% probability to diversify broker exploration. This path is disabled for the nine non-MQTT primary targets, so the primary lineage results do not depend on node-stagnation randomization.

Plateau detection uses the IPSM edge growth rate $\dot{E}$, measured as edges gained per minute. LoopFuzz recalculates this rate every 60 seconds of wall-clock time and uses it to set an adaptive threshold:

$$\tau(\dot{E}) = \begin{cases} 700 & \text{if } \dot{E} > 5 \\ 600 & \text{if } 1 < \dot{E} \leq 5 \\ 512 & \text{if } \dot{E} \leq 1 \end{cases}.$$

The initial value is τ=512, aligned with the ChatAFL plateau floor. Textual and MQTT runs use the same adaptive range in the reported implementation; Mosquitto remains outside primary performance conclusions. Plateau triggering is therefore a rate-sensitive control decision, not a fixed-frequency query rule.

The update is immediate. Every 60 seconds, the implementation recomputes $\dot{E}$, overwrites τ, and tests the current unproductive streak against the new threshold in the same control path. Changing τ does not reset the streak, and there is no hysteresis or minimum dwell time. The streak resets only on an interesting queue path or plateau entry, so a downward threshold update can immediately open the plateau gate. This is the reported implementation semantics, bounded by the 60-second cadence and $\{512, 600, 700\}$ threshold set.

Assistance is invoked only when edge growth remains weak and fewer than 64 model interventions have occurred. Inside the plateau path, hypotheses are initialized lazily, local repair remains gated by validation count and accumulated counterexamples, and the prompt reuse filter skips repeated plateau-context hashes rather than replaying model outputs.

The plateau prompt exposes a compact state summary: IPSM size, edge-growth rate, favored seeds, queued paths, current target state, and least productive states. A model can

Table 5. Auditable $h(s)$ update rules

Stage	Condition	Assignment	Interpretation
Initialization	Text-protocol adapters classify by $\lfloor s/100 \rfloor$; class 4/5 means likely rejection. Others use behavioral fallback.	$h(s) = 1$ for 4xx/5xx text-protocol states; otherwise $h(s) = 0$.	Structural response-code prior; no binary/bit-field rejection-class map.
Behavioral promotion	$h(s) = 1$, $N_s \geq 10$, and $p_s = D_s/(N_s + 1) \geq 0.1$.	$h(s) \leftarrow 2$.	Rejection-coded state becomes fully schedulable after repeated queued-path yield.
Behavioral demotion	$h(s) = 0$, $N_s \geq 30$, and $p_s < 0.005$.	$h(s) \leftarrow 1$.	Unknown state becomes likely error after repeated low-yield targeting.
No other update	All other cases, including productive $h = 0$, sub-threshold $h = 1$, and all $h = 2$.	$h(s)$ unchanged.	No $0 \rightarrow 2$, $1 \rightarrow 0$, $2 \rightarrow 1$, streak counter, or decay; $h = 1$ recovers only via $1 \rightarrow 2$.

redirect the target state, reprioritize a small seed set, or propose bounded mutations over an existing seed. Request-producing actions still need runtime admission, while scheduling-only actions pass schema and identifier checks and are tested by later executions. The LLM remains a state-directed policy proposer rather than an oracle.

MQTT-specific scheduling is treated as a protocol adaptor, not the general LoopFuzz mechanism. It uses packet-type and mutation-family controllers to expose broker forwarding behavior to the IPSM; a lightweight oracle can also log security indicators. These signals support auditing and triage, but they are outside the primary non-MQTT performance evidence and are not treated as confirmed vulnerabilities without normalized crash/root-cause review.

Algorithm 1 specifies the plateau path and the hypothesis-repair gate. It formalizes how state-aware triggering, runtime admission checks, and counterexample-guided local repair interact within one campaign loop.

Algorithm 1: Runtime-Validated Plateau Recovery

Input: Target T, LLM $\mathcal{L}$, corpus C, edge-growth $\dot{E}$
Output: Coverage bitmap $\mathcal{B}_{global}$ and IPSM $\mathcal{M}$

Initialize $\mathcal{B}_{global}$ and $\mathcal{M}$
while *the campaign budget remains* **do**
  $s \leftarrow \textsc{SelectSeed}(C, \mathcal{M})$
  $(trace, Q) \leftarrow \textsc{MutateExecute}(T, s)$
  Update $\mathcal{B}_{global}$, $\mathcal{M}$, and frontier scores from $(trace, Q)$
  if *sampled validation is due* **then**
    Validate hypotheses on parsed regions; append failures and update fitness
  end
  if *the adaptive plateau gate opens* **then**
    Initialize hypotheses on first use, if needed
    if *accumulated counterexamples and low fitness open the repair gate* **then**
      Patch the lowest-fitness hypothesis locally
    end
    $c \leftarrow \textsc{BuildPrompt}(\mathcal{M}, C, Q, \dot{E})$
    if *c is not in the recent prompt cache* **then**
      $r \leftarrow \mathcal{L}(c)$ under process isolation
      Run parsed request candidates as bounded trials if P holds
      Queue requests only when post-trial U, R, and G hold
      Apply scheduling-only actions after schema and identifier checks
      Record failed request trials as counterexample or productivity evidence
    end
  end
end

Thus, sampled validation accumulates evidence rather than directly triggering repair. Repair is reached only through the plateau path when validation-count, low-fitness, and accumulated-counterexample gates hold. Initial enrichment, lazy hypothesis construction, process-isolated model calls, and bounded network cleanup are operating conditions rather than separate novelty claims.

5. Experimental Evaluation Design

We organize the evaluation around three research questions (RQs) that match the evidence boundary while separating metrics. The main experiment is a lineage experiment: it asks whether the integrated LoopFuzz controller changes a campaign that already has response-derived IPSM feedback and ChatAFL-style LLM proposals. Because the archive lacks a *LoopFuzz-no-admission* arm, the RQs do not isolate the complete $P/U/R/G$ boundary from local repair, frontier scheduling, and adaptive triggering. NSFuzz/MBFuzzer rows and policy perturbations are diagnostic context, not same-pipeline rankings.

- **RQ1 (Wall-Clock Branch-Coverage Accumulation):** Under the same selected-run and container-quota setting, does LoopFuzz accumulate branch coverage earlier or over a larger area under the branch-coverage curve than AFLNet and the controlled ChatAFL-lineage baseline?
- **RQ2 (Response-Derived State Exploration):** Does the evaluated integrated policy sustain response-derived IPSM growth, as measured by final `edges` and IPSM-edge trajectories, relative to the AFLNet/ChatAFL lineage baselines?
- **RQ3 (Robustness and Boundary Behavior):** Are the observed gains stable across repeated runs, or do variance, coefficient of variation, saturated targets, and negative targets reveal limits of the current controller?

5.1. Experimental Setup

We ran all experiments on a dedicated Ubuntu 20.04 server with 32 central processing unit (CPU) cores and 64 GB memory. The experimental material spans 10 implementations across seven protocol families, but only the first nine targets below form the primary AFLNet/ChatAFL/LoopFuzz benchmark:

1. **FTP:** LightFTP, bftpd, ProFTPD, and Pure-FTPd.
2. **SMTP:** Exim.
3. **RTSP:** Live555.
4. **SIP:** Kamailio.
5. **DAAP/DACP:** Forked-daapd.
6. **HTTP:** Lighttpd1.

7. **MQTT:** Mosquitto, retained as an auxiliary MQTT diagnostic case rather than as a primary benchmark target.

The primary comparison uses AFLNet, a controlled ChatAFL-lineage baseline, and LoopFuzz on the nine non-MQTT targets. AFLNet supplies the response-derived IPSM lineage; ChatAFL supplies the direct LLM-assisted predecessor. The ChatAFL row is a same-harness, same-target-lineage reimplementation that preserves ChatAFL's template extraction, seed enrichment, and open-loop plateau roles while using this study's controlled endpoint and archive pipeline. This reduces model-provider confounding but is not a direct reproduction of the original `gpt-3.5-turbo`-based ChatAFL results. Endpoint and trajectory results come from the primary selected-run archive, while policy ablations come from a separate ablation archive. Mosquitto is reported as an auxiliary diagnostic: the Mosquitto summary now contains 10 AFLNet runs and eight LoopFuzz runs, but it still lacks a controlled ChatAFL cell and a same-metric MBFuzzer cell.

The equal-budget primary subjects are textual or semi-textual request-response protocols, matching the setting where LoopFuzz's local parsers, response classifiers, and response-derived IPSM feedback are available. The primary benchmark does not include TLS, DTLS, QUIC, SSH, IPsec, or other opaque binary/encrypted targets; applying the policy to those protocols would require target-specific adapters before comparable evaluation.

For the controlled lineage comparison, ChatAFL and LoopFuzz use the same OpenAI-compatible CCTQ chat-completion relay rather than the original public `gpt-3.5-turbo` surface. The implementation calls `https://www.cctq.ai/v1/chat/completions` with `model=gpt-5.4-mini`; authentication uses the `KEY` environment variable and no key is stored. The archive does not record an immutable provider snapshot or version date, so the model path is auditable through implementation, run summaries, raw plateau replies, aggregate token counters, and failure handling, but not bit-for-bit reproducible through the public OpenAI service.

Call configuration is fixed by fuzzer and call class. Controlled ChatAFL uses temperature 0.5 for templates/enrichment and 1.5 for open-loop plateau generation; LoopFuzz uses 0.5 for inherited templates/enrichment, 0.3 for hypothesis generation and repair, 1.2 for plateau assistance, and optional MQTT checks at 0.2. The controlled ChatAFL token cap is `max_tokens`=2048, while LoopFuzz uses 4096. Neither implementation sends a model-side seed, `top_p`, presence penalty, or frequency penalty; provider defaults therefore remain a reproducibility boundary.

Transport and failure policy are fixed: 30-second connection timeout, 120-second total request timeout, low-speed abort below one byte per second for 60 seconds, two-second sleep between non-authentication retries, and plateau-helper kill after 60 seconds. Missing credentials, exhausted retries, malformed structured output, schema failure, or helper timeout produce no admitted model action. System prompts are fixed by call class; protocol name, examples, responses, and state summaries are contextual inputs rather than hand-specialized target prompts.

Because plateau calls use nonzero temperature, the prompt reuse filter is a repetition guard rather than deterministic memoization. It hashes the stalled context, keeps the most recent 64 hashes, and skips repeated contexts without storing prior model outputs. Raw plateau replies and aggregate token counters are retained, but per-call prompt/payload/latency/action ledgers are not; Table 7 summarizes the remaining replay boundary.

Resource isolation and concurrency.. Each long-running sample is launched in an isolated ProfuzzBench-style container. In the standard archive, each container uses a one-CPU quota, an 8GB memory limit, an 8GB swap limit, and an initialization wrapper; the controller records the container identity, waits for completion, and recovers one compressed artifact per run. Runs are quota-limited but not pinned to physical cores.

Within a target-fuzzer invocation, simultaneous containers equal `NUM_CONTAINERS`. The aggregate artifact preserves selected terminal summaries but not the global batch schedule, so co-scheduling cannot be reconstructed. The selected tar archives do preserve AFLNet-style `fuzzer_stats` and `plot_data` counters, including `execs_done`, `execs_per_sec`, `slowest_exec_ms`, and `peak_rss_mb`; they do not preserve cgroup CPU usage, throttling, host-load, storage, network, daemon-reset, or batch-schedule traces. Table 10 summarizes why wall-clock curves are descriptive same-quota traces, not CPU-time-normalized throughput claims.

Port conflicts are avoided by container network namespaces. Non-MQTT services bind inside each container, and MQTT artifacts use separate container networks or generated worker endpoints. The earlier short Mosquitto LoopFuzz rows are not used; Table 22 reports the later Mosquitto selected-summary data, which remain outside the primary family because the auxiliary surface is not a complete AFLNet/controlled-ChatAFL/LoopFuzz matrix.

Campaigns are bounded by a fixed wall-clock wrapper with a short kill grace and preserved exit status. Target reset is handled by the shared AFLNet-lineage harness and, where available, target-specific cleanup hooks. Offline coverage replay restarts or kills daemons with bounded cleanup commands before accumulating branch summaries. The launcher does not set a fixed fuzzer random seed, and the archive does not preserve per-run random seeds. We therefore treat repeated runs as independent stochastic trials rather than bit-for-bit replays.

Run filtering and missing-data handling.. The intended primary matrix is 10 runs per fuzzer-target pair. We retain completed runs with at least 1400 archived minutes and final `b_abs`/IPSM `edges`; if more than 10 rows remain, the lowest 10 run identifiers are kept before endpoint inspection. All nine non-MQTT target-fuzzer cells in the selected cache contain 10 eligible rows. This proves selected-cache completeness only: the package lacks an immutable pre-filter launch ledger, so failed launches or artifact-recovery failures outside the selected cache cannot be enumerated. Tables 8 and 9 separate these two claims.

5.2. Baselines, Metrics, and Statistical Protocol

We use two baseline layers. The primary baselines are AFLNet and the controlled ChatAFL-lineage reimplementation; the auxiliary baseline is NSFuzz [33], with Mosquitto artifacts reported separately for AFLNet, LoopFuzz, and MBFuzzer. Because the auxiliary artifacts expose different metric surfaces, their tables are diagnostics rather than same-pipeline rankings. External competitiveness would require rerunning NSFuzz/MBFuzzer under shared builds, instrumentation, replay, budgets, and metric extraction, or converting LoopFuzz to their metric surfaces.

Table 6. Controlled ChatAFL-lineage baseline scope

Aspect	Original ChatAFL	Controlled ChatAFL	LoopFuzz
Role	LLM-guided artifact.	Same harness; not reproduction.	Closed-loop control.
Model/interface	OpenAI; `gpt-3.5-turbo`; user key.	CCTQ; `gpt-5.4-mini`; `KEY`.	Same CCTQ/model.
Temperature	T=0.5 grammar/enrich.; 1.5 plateau.	T=0.5 template/enrich.; 1.5 plateau.	T=0.5 template/enrich.; 0.3 hyp./repair; 1.2 plateau; 0.2 MQTT.
Calls/max tokens	5 grammar; cap unspecified.	5 grammar/enrich.; 2 plateau; 2048.	5 grammar/enrich.; 3 hyp./repair; 2 plateau; 1 MQTT; 4096.
Control	Templates; seed enrich.; open-loop.	ChatAFL prompts; no $P/U/R/G$/repair.	Structured actions; validation; $P/U/R/G$; frontier; repair.
Reproducibility	Public model state.	Endpoint changed; lineage only.	Summaries/replies/tokens; no bit replay.

Table 7. LLM stochasticity and replay-accounting status

Field	Status	Core evidence or gap
Sampling params	Partial	Records `model`, `messages`, `max_tokens`, temperature; no `top_p`, penalties, or `seed`.
Provider seed	No	No seed requested; CCTQ seed support unverified.
Prompt payloads	No	In memory only; no per-call ledger.
Replies	Partial	Plateau replies in `stall-interactions/llm-suggest-*`; other classes incomplete.
Parsed actions	Partial	Validation exists; no per-action outcome/effect ledger.
Tokens	Aggregate	Run/sample totals only; no per-call sidecars.
Latency/timeouts	No	Constants known; no per-call timing/timeout rows.
Action replay	No	No provider-free recorded-action stream.

Table 8. Selected-cache run inventory

Target	AFLNet selected rows	ChatAFL selected rows	LoopFuzz selected rows
LightFTP	10 selected	10 selected	10 selected
bftpd	10 selected	10 selected	10 selected
ProFTPD	10 selected	10 selected	10 selected
Pure-FTPd	10 selected	10 selected	10 selected
Exim	10 selected	10 selected	10 selected
Live555	10 selected	10 selected	10 selected
Kamailio	10 selected	10 selected	10 selected
Forked-daapd	10 selected	10 selected	10 selected
Lighttpd1	10 selected	10 selected	10 selected

Table 9. Primary launch-ledger auditability gaps

Field	Value	Boundary
Planned rows	Intended 270	Design only (9×3×10); no launch ledger.
Launched runs	Not recoverable	No pre-filter manifest, container identifiers, or failed-start rows.
Selected completed rows	270	Retained after filtering.
Excluded < 1400 min	0 selected; pre-filter unknown	None in selected cache; pre-filter unknown.
Missing endpoint metrics	0 selected; pre-filter unknown	Selected rows have final `b_abs`/`edges`.
Artifact/recovery failure	Not recoverable	No missing-archive/extraction/`fuzzer_stats` failure rows.
Endpoint-table rows	270	Selected-cache complete; survivorship risk remains.

Table 10. Wall-clock fairness and resource-trace gaps

Quantity	Status	Gap
CPU/memory quota	Static	1 CPU, 8GB memory, 8GB swap; no core/time trace.
Cgroup CPU/throttling	No	No cgroup, throttling, frequency, or host-load trace.
Terminal exec rate	Yes	`fuzzer_stats` rows; no mutation/LLM/trial split.
Trajectory exec rate	Partial	`plot_data`; no resource-normalized throughput.
Coverage/execution	Partial	`b_abs`/`execs_done` computable; no CPU/path attribution.
Daemon reset/cleanup	No	Hooks known; counts, failures, and latency absent.
Batch/co-scheduling	No	No global overlap schedule.
Host input/output timing	No	No storage, latency, or jitter trace.

The primary endpoint variables are final `b_abs` and final `edges`. For AFLNet, ChatAFL, and LoopFuzz, `b_abs` is the absolute branch count from the same selected-summary and coverage-replay pipeline, and `edges` is the archived response-derived IPSM edge count. To keep the RQs distinct, RQ1 uses branch trajectories, RQ2 uses final and trajectory-level IPSM edges, and RQ3 uses dispersion and boundary cases. NSFuzz branch/edge fields and MBFuzzer Mosquitto branches remain external artifact counts, not AFLNet-lineage metrics.

For each primary target and endpoint metric, we summarize selected endpoint values with mean and sample standard deviation (SD). We also compute pairwise two-sided Mann-Whitney U tests comparing LoopFuzz against AFLNet and ChatAFL, followed by Benjamini–Hochberg (BH) false discovery rate (FDR) control across the 36 primary pairwise tests (nine targets, two metrics, and two baselines). Cliff's δ reports effect size, with positive values favoring LoopFuzz. We use the per-run terminal vectors in the archived selected summaries, after applying the same deterministic selection rule as Table 15. Exact tests are not always available because several rows contain ties, so the implementation uses SciPy's tied-rank handling. To quantify endpoint uncertainty, we also compute percentile bootstrap 95%

confidence intervals for the mean difference, LoopFuzz minus baseline, using a fixed random seed. These intervals are descriptive and are not multiplicity-adjusted; they complement the rank tests by showing the plausible mean-difference range under the selected-run sample.

The primary design has $n = 10$ selected runs per fuzzer-target cell, so the corrected tests have limited power for medium effects. In the archived 36 primary endpoint tests, every BH-corrected discovery has $|\delta| \geq 0.57$, while several positive-by-mean rows with medium or large Cliff effects remain at $q > 0.05$. We therefore interpret such rows as *inconclusive under current power*, not as evidence of no effect. Table 17 reports the positive-by-mean, not-corrected rows where this distinction matters most. A larger repeated-run archive would be needed to turn these rows into stable acceptance or rejection statements under the same FDR protocol.

The trajectory metrics are descriptive because the cached trajectory tables preserve 60-minute mean curves rather than full per-run curve vectors. RQ1 therefore reports two mean-trajectory quantities: the trapezoidal branch-coverage area under the curve over 0–1440 minutes, expressed as LoopFuzz/AFLNet and LoopFuzz/ChatAFL ratios, and T_{95}, the first interpolated minute at which a mean branch-coverage curve reaches 95% of the best final mean `b_abs` among the three primary systems for that target. RQ3 reports coefficient of variation, CV = SD/mean, for endpoint stability. Crash and vulnerability discovery are not part of these endpoint hypothesis tests because cross-fuzzer crash triage and root-cause validation are not normalized. Section 6.5 therefore reports a separate preliminary triage for the selected LoopFuzz artifacts; it should be read as security evidence from those fuzzer-target cells, not as a vulnerability-yield ranking for LoopFuzz.

5.3. Policy-Ablation Design

The ablation archive exposes four policy switches: counterexample-guided local repair, frontier scheduling, hypothesis validation, and adaptive triggering. It does not include a full no-admission variant. A *LoopFuzz-no-admission* design would preserve the same model calls, prompts, query budget, plateau triggers, scheduling policy, and schema checks, but bypass the $P/U/R/G$ admission checks and directly enqueue request-producing model suggestions. Table 26 therefore measures sensitivity to exposed sub-policies, not the complete admission boundary, and should not be read as evidence that the admission formula alone caused the endpoint gains. We apply the same deterministic run-selection rule as the primary comparison: keep completed runs with at least 1400 minutes, and if more than 10 valid runs remain, keep the lowest 10 run identifiers. Because this archive is separate from the primary selected-run archive, its full-controller column need not match every primary-table endpoint exactly. All non-MQTT full-controller cells contain 10 eligible completed runs. Among non-MQTT perturbations, the four ProFTPD cells contain nine after excluding one near-limit non-completed row in each perturbation. The auxiliary Mosquitto ablation now contributes eight full-controller rows and 10 rows for each perturbation.

For each exposed perturbation and endpoint metric, Table 27 compares the perturbation against its local full-controller cell with a two-sided Mann–Whitney U test. We report Benjamini–Hochberg q values across all 80 ablation-vs-full endpoint comparisons, Cliff's δ with direction "perturbation minus full," and percentile bootstrap 95% confidence intervals for the mean difference using a fixed random seed. These tests are exploratory sensitivity diagnostics for the separate ablation archive. They are not confirmatory evidence of per-module necessity, and small uncorrected deltas are not interpreted as mechanism trends unless the corrected inference supports them.

5.4. Trace Accounting and Cost Reporting

Because LoopFuzz is a closed-loop LLM-in-the-loop system, terminal coverage alone is not enough for mechanism accounting. The selected LoopFuzz archives preserve run-level fuzzer statistics and raw plateau replies, which recover model-call counts, plateau-call counts, token counters, prompt-reuse hits, and sampled hypothesis-validation counters. They do not normalize each model proposal into a row with extracted candidate, $P/U/R/G$ decision, rejection reason, bounded-trial response, coverage/IPSM gain, repair event, and latency. Table 11 therefore reports missing candidate-level quantities explicitly, and we do not infer admission ratios or avoided queue pollution from run-level counters.

Runtime overhead is the second missing mechanism layer. Exact trial overhead would require trial timestamps, trial counts, timeout flags, and separation of ordinary mutation from LLM waits and bounded admission trials. The selected archives preserve terminal fuzzer execution counters, but not this decomposition. The only defensible upper bound concerns model-call waiting time. Let C_t be the mean archived LoopFuzz LLM-call count for target t and P_t be the mean plateau-call count from Table 13. With 120 seconds for non-plateau requests and 60 seconds for plateau helpers, a conservative timeout upper bound is

$$T_{\mathrm{LLM}}^{\max}(t) = 120(C_t - P_t) + 60P_t \quad \text{seconds per run.}$$

This is an upper bound, not an observed latency: calls that returned normally consumed less time, and the bound excludes bounded-trial execution time because trial timestamps are unavailable.

Table 13 reports the recoverable run-level mechanism counters. Because candidate-level admission logs are absent, these values are not admission rates, repair-effectiveness measurements, overhead summaries, cost-efficiency claims, or semantic-drift prevalence estimates.

Interpreting low hypothesis-pass rates.. The low `Hyp. pass` values in Table 13 require a narrow interpretation. This column measures sampled validation of maintained grammar and field hypotheses against parsed execution evidence. It is not the P predicate, not a post-trial $U/R/G$ pass rate, and not the fraction of LLM-generated request candidates admitted into the durable queue. A target with 0.0±0.0% `Hyp. pass` therefore indicates that the sampled hypothesis-validation oracle rarely accepted the maintained grammar hypothesis in those archived samples; it does not imply that no request-producing candidate passed runtime admission or that no scheduling hint affected the campaign.

The same archive lacks repair-trigger events, patch identifiers, before/after fitness deltas, post-repair admission outcomes, and post-repair coverage/IPSM gains. Given the low sampled pass rates, we treat hypothesis validation and local repair as auditable mechanisms and possible contributors, not as independently verified primary drivers.

Table 11. Candidate-level admission-accounting gaps

Quantity	**Needed fields**	**Status**	**Implication**
LLM request proposals	Prompt/reply id; candidate id; action; target/run.	Missing.	No admission denominator.
P fail count	Schema/parse flag; parse error; stop reason.	Missing.	No parse-failure rate.
U fail count and rejection reasons	Response code/bytes; timeout/reset; rejection class.	Missing.	No rejection or drift rate.
R fail count	Source/destination states; scheduled frontier; reachability.	Missing.	No non-moving rate.
G fail count	Coverage delta; IPSM novelty; favored flag; promotion.	Missing.	No valid-but-no-gain rate.
Admitted count and admission ratio	Promotion flag plus proposal denominator.	Missing.	No admission ratio.
Median bounded-trial latency	Trial start/end; timeout flag.	Missing.	No admission-overhead summary.
Median coverage/IPSM gain	Branch/edge delta; IPSM novelty; promotion.	Missing.	No candidate-to-gain link.

Table 12. Overhead-accounting gaps and timeout upper bounds

Overhead item	**Status**	**Available bound**	**Boundary**
Bounded-trial count	No	No post-P trial rows or candidate denominator.	Admission vs mutation not separable.
Bounded-trial latency/time	No	No trial start/end times or timeout flags.	Wall-clock only; no normalization.
LLM-call latency	Bounded only	Timeout caps: 120 s non-plateau; 60 s plateau.	Upper bounds, not observations.
Total time in LLM path	Upper-bound only	Target-mean range: 1.42–8.98 h/run (Kamailio min; Live555 max).	Loose maxima; no trial time.
Terminal executions/sec	Recoverable	`fuzzer_stats/plot_data`; no path split.	Descriptive wall-clock only.
Coverage per million executions	Computable	Final `b_abs/execs_done`; mixed paths.	No CPU/trial-efficiency claim.

Table 13. LoopFuzz run-level mechanism counters

Target	LLM calls	Plateau calls	Prompt tok. (k)	Compl. tok. (k)	Hyp. checks	Hyp. pass	Hyp. fitness
LightFTP	151.1±7.6	64.0±0.0	164.0±191.1	15.5±3.3	1162.1±177.9	0.0±0.0%	0.08±0.01
bftpd	246.6±3.9	64.0±0.0	154.4±11.3	24.9±1.6	891.5±115.9	5.4±12.0%	0.17±0.21
ProFTPD	196.7±9.1	14.1±3.5	86.5±3.9	17.1±0.9	436.8±111.0	2.7±5.2%	0.11±0.10
Pure-FTPd	228.2±6.2	64.0±0.0	474.2±550.9	21.9±1.4	1293.0±239.6	8.4±13.6%	0.16±0.17
Exim	105.4±13.1	15.5±5.9	49.5±16.4	9.1±2.6	266.1±120.8	0.0±0.0%	0.10±0.02
Live555	301.1±66.7	63.5±1.6	210.5±40.0	171.4±45.3	754.2±675.4	0.0±0.0%	0.18±0.17
Kamailio	51.3±23.9	17.5±8.8	63.2±39.0	51.1±29.3	3928.4±1963.2	0.0±0.0%	0.10±0.02
Forked-daapd	59.9±35.7	24.4±11.4	86.4±77.5	74.7±72.8	1722.8±144.5	0.0±0.0%	0.14±0.02
Lighttpd1	165.2±25.8	60.7±6.5	148.5±29.9	34.4±9.4	2715.3±1805.1	0.0±0.0%	0.12±0.03

Table 14. Mechanism-counter variability diagnostics

Target	Tok. range (k)	Max-token run: `b_abs/edges`	Hyp. active; range	Max-hyp run: `b_abs/edges`	ρ_{tok} b/e	ρ_{hyp} b/e
LightFTP†	66.3–714.8	r10: 72/232	8/10; 0–1345	r1: 71/204	$+0.00/+0.15$	$-0.08/-0.82$
bftpd	162.7–198.7	r1: 494/213	10/10; 635–1032	r5: 509/198	$-0.08/-0.53$	$+0.22/-0.12$
ProFTPD	98.1–110.0	r4: 5382/264	8/10; 0–651	r9: 5191/256	$+0.36/-0.35$	$+0.17/-0.14$
Pure-FTPd	146.8–1349.6	r8: 1232/293	6/10; 0–1611	r4: 1242/295	$-0.55/+0.12$	$+0.15/-0.10$
Exim	32.0–82.5	r6: 3887/131	7/10; 0–389	r2: 3808/142	$-0.18/+0.09$	$-0.17/+0.18$
Live555	268.9–481.0	r8: 3077/150	8/10; 0–1746	r9: 3062/155	$+0.56/-0.41$	$-0.35/-0.18$
Kamailio	0.0–179.6	r2: 10261/134	7/10; 0–6106	r4: 10252/138	$-0.35/+0.54$	$+0.13/+0.79$
Forked-daapd	6.5–394.8	r5: 2011/20	6/10; 0–1950	r10: 2134/23	$-0.07/-0.83$	$-0.09/-0.39$
Lighttpd1	118.0–245.0	r2: 2266/41	10/10; 125–6254	r9: 2327/47	$+0.59/+0.72$	$+0.13/+0.65$

Table 14 explains why the large SDs in Table 13 should be read as run-to-run mechanism variability rather than as a stable treatment effect. Pure-FTPd has three selected runs with unusually large total token volume (runs 8–10 exceed 1.2M total tokens), but the largest-token run does not have the largest final `b_abs` or `edges`. LightFTP has one large token outlier, yet branch coverage is saturated and the highest-token run is not a meaningful branch-coverage outlier. Kamailio and Lighttpd1 show the largest hypothesis-check ranges; their maximum-check runs have high edge endpoints, but the within-target correlations are descriptive and based on only 10 runs. In short, the current archive does not support a monotone claim that more tokens or more sampled hypothesis checks produce better endpoints.

Token outliers are plausible because provider-reported `usage` accumulates across template extraction, seed enrichment, lazy hypothesis generation, repair, and plateau assistance, and plateau prompts embed run-dependent examples, history, and state context. They are not explained by repeated identical plateau contexts because prompt-reuse hits were 0.0±0.0 in selected primary LoopFuzz cells. The archive lacks retry logs

and prompt-length breakdowns by call class, so token counts are audit counters only; no dollar cost or cost-efficiency ranking is reported.

The plateau-call counts in Table 13 vary substantially across targets, so they should not be read as a uniform treatment dose. Plateau activation is an endogenous controller outcome: a target can receive few plateau calls because ordinary mutation and state scheduling continue to produce progress, because the adaptive gate is conservative, or because the model budget is consumed by non-plateau paths such as template extraction, seed enrichment, hypothesis generation, or repair. Section 6.4 therefore analyzes the results by high- and low-activation strata instead of pooling all targets as if they received the same runtime LLM exposure.

6. Results

Table 15 summarizes the terminal aggregates used for the primary AFLNet and ChatAFL lineage comparison from the primary selected-run archive. Figs. 3 and 4 visualize the same selected runs as per-target trajectories. This layout follows the protocol-level comparison view: each panel keeps one target on its own scale, so the curves show both endpoint separation and the time at which each fuzzer saturates. LightFTP is explicitly marked as saturated for branch coverage: its `b_abs` panel is retained for transparency, but it is not used as evidence for coverage breadth. Table 16 reports the corrected endpoint rank-test summary. The RQ analysis below deliberately separates the evidence surfaces: Table 18 answers RQ1 with branch-coverage accumulation metrics, including branch area under the curve (AUC), where NR denotes not reached within 1440 minutes; Table 15 and Fig. 4 answer RQ2 with response-derived IPSM edges; and Table 19 answers RQ3 with run-to-run dispersion and boundary cases.

The results are not a mechanism-complete admission analysis. Tables 11, 12, and 13 separate missing candidate-level $P/U/R/G$ fields, unavailable trial-overhead accounting, and recoverable run-level counters. Accordingly, Fig. 1 remains illustrative and the run-level counters are not interpreted as admission rates, rejection distributions, repair-latency summaries, throughput-normalized measurements, or queue-pollution measurements.

6.1. RQ1: Wall-Clock Branch-Coverage Accumulation

RQ1 asks about coverage accumulation over the 24-hour wall-clock campaign, not only the terminal `b_abs` endpoint. Table 18 reports two descriptive metrics from the cached mean branch-coverage trajectories: T_{95} to a target-specific common threshold and the 0–1440 minute branch-AUC ratio. This separates early or sustained coverage accumulation from the endpoint hypothesis tests in Table 16. The interpretation is deliberately limited to the selected same-container-quota wall-clock surface. Table 10 shows that the archive has fuzzer execution-rate counters, but not the CPU/resource traces needed to convert these trajectories into CPU-time-normalized efficiency claims.

LoopFuzz reaches the T_{95} threshold earlier than both lineage baselines on seven of nine primary targets: bftpd, ProFTPD, Pure-FTPd, Exim, Live555, Kamailio, and Lighttpd1. On Live555 and Lighttpd1, neither AFLNet nor ChatAFL reaches the threshold by 1440 minutes, while LoopFuzz reaches it at 57 and 270 minutes, respectively. The branch-AUC ratios show the same broad pattern: LoopFuzz has a larger mean branch-coverage area than both baselines on eight targets. The exception is Forked-daapd, where LoopFuzz does not reach the common T_{95} threshold and its branch-AUC is below AFLNet and slightly below ChatAFL.

Two cautions limit RQ1. LightFTP is saturated, so its roughly 1% AUC advantage is weak branch-efficiency evidence. Also, ChatAFL is not a uniform branch-coverage regression relative to AFLNet: it reaches T_{95} earlier on bftpd, ProFTPD, Pure-FTPd, and Kamailio, but later on Exim and Forked-daapd. Thus LoopFuzz improves descriptive wall-clock branch accumulation on most primary lineage targets under the same selected container-quota setting, but not CPU-time-normalized superiority.

6.2. RQ2: Response-Derived State Exploration

RQ2 uses the state-exploration surface rather than branch-coverage accumulation. Terminal `edges` provide the clearest primary-lineage evidence for improved response-derived IPSM exploration. In the AFLNet/ChatAFL comparison, LoopFuzz has higher mean final `edges` than both lineage baselines on the nine primary targets, as visualized in Fig. 4. After BH correction, the LoopFuzz-versus-AFLNet edge advantage remains significant on eight targets; Forked-daapd remains positive by mean but inconclusive under the current $n = 10$ power. Against ChatAFL, the corrected edge advantage remains significant on six targets. Kamailio and Forked-daapd have positive edge means with medium-to-large Cliff effects but do not pass BH correction, so Table 17 treats them as power-sensitive follow-up rows. Lighttpd1 is different: its edge difference against ChatAFL is positive by mean but small and high-variance.

This state-exploration result should not be collapsed into the RQ1 coverage result. Forked-daapd is the clearest separation: LoopFuzz has the highest mean response-derived `edges` but lower and more variable branch coverage. Lighttpd1 separates the metrics in the other direction: LoopFuzz improves branch coverage, while the edge comparison against ChatAFL is positive by mean but not corrected-significant and has high edge dispersion in Table 19. Thus, response-derived IPSM growth is a useful companion diagnostic, not a universal coverage proxy; Section 7.1 makes this boundary explicit with an endpoint-level edge-to-branch diagnostic.

Table 21 keeps the NSFuzz edge archive separate from the AFLNet-lineage state-exploration metric. AFLNet, ChatAFL, and LoopFuzz report response-derived IPSM edges, whereas the NSFuzz column is an artifact edge count from a different state-aware implementation. The diagnostic mismatch is visible in the data: on LightFTP, NSFuzz has a much larger branch-count artifact in Table 20 but an artifact edge count of only 11.0±0.0; on Forked-daapd, NSFuzz reports 27.0±6.1 artifact edges while the AFLNet-lineage edge counts are 21.1±1.3, 20.5±2.1, and 22.1±1.5.

6.3. RQ3: Robustness and Boundary Behavior

RQ3 asks whether the observed gains are stable across repeated runs and where the controller's evidence breaks down. Table 19 reports endpoint CVs. LoopFuzz has the lowest branch CV on bftpd and Pure-FTPd, matches or stays close to the lowest branch CV on Exim and Live555, and still improves mean branch coverage on those targets. These are the strongest stability-compatible cases for the current archive.

Table 15. Primary endpoint `b_abs` and `edges`

Proto.	Target	Final `b_abs`			Final `edges`		
		AFLNet	ChatAFL	LoopFuzz	AFLNet	ChatAFL	LoopFuzz
FTP	LightFTP†	71.0±0.0	71.0±0.0	**71.7±1.6**	182.8±10.2	194.2±10.2	**216.6±10.4**
FTP	bftpd	488.6±13.9	492.3±15.1	**501.5±7.4**	166.9±9.4	205.2±9.7	**221.7±16.9**
FTP	ProFTPD	4919.9±97.8	5317.3±101.7	**5358.8±136.1**	180.5±23.1	250.5±9.1	**271.7±18.8**
FTP	Pure-FTPd	1054.3±76.4	1179.2±31.0	**1240.5±25.8**	213.9±11.9	266.2±14.3	**299.6±26.2**
SMTP	Exim	3794.5±42.4	3746.2±47.7	**3908.5±43.2**	88.3±5.3	116.4±10.2	**128.5±7.8**
RTSP	Live555	2894.1±23.3	2911.1±10.9	**3088.7±22.4**	103.9±10.1	151.2±3.9	**158.7±8.5**
SIP	Kamailio	10244.0±118.1	10314.7±98.4	**10402.7±141.3**	101.1±7.7	115.7±13.9	**127.6±9.6**
DAAP/DACP	Forked-daapd	**2326.3±77.3**	2315.3±50.1	2171.9±226.0	21.1±1.3	20.5±2.1	**22.1±1.5**
HTTP	Lighttpd1	1930.4±18.7	1987.5±38.3	**2156.5±111.8**	14.9±3.8	29.7±4.7	**32.5±10.8**

Table 16. LoopFuzz pairwise endpoint statistics

Target	`b_abs`		`edges`	
	vs. AFLNet	vs. ChatAFL	vs. AFLNet	vs. ChatAFL
LightFTP†	0.078/0.093/ + 0.30	0.078/0.093/ + 0.30	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 0.94
bftpd	0.017/0.030/ + 0.64	0.150/0.169/ + 0.39	< 0.001/ < 0.001/ + 1.00	0.031/0.043/ + 0.58
ProFTPD	< 0.001/ < 0.001/ + 1.00	0.623/0.623/ + 0.14	< 0.001/ < 0.001/ + 1.00	0.012/0.022/ + 0.67
Pure-FTPd	< 0.001/ < 0.001/ + 1.00	< 0.001/0.002/ + 0.90	< 0.001/ < 0.001/ + 1.00	0.005/0.009/ + 0.76
Exim	< 0.001/0.002/ + 0.90	< 0.001/ < 0.001/ + 0.98	< 0.001/ < 0.001/ + 1.00	0.025/0.037/ + 0.60
Live555	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 1.00	0.034/0.045/ + 0.57
Kamailio	0.021/0.035/ + 0.62	0.345/0.365/ + 0.26	< 0.001/ < 0.001/ + 0.97	0.049/0.063/ + 0.53
Forked-daapd	0.026/0.037/ − 0.60	0.026/0.037/ − 0.60	0.164/0.179/ + 0.37	0.113/0.131/ + 0.42
Lighttpd1	< 0.001/ < 0.001/ + 1.00	< 0.001/ < 0.001/ + 0.95	< 0.001/ < 0.001/ + 0.98	0.596/0.613/ + 0.15

Table 17. Power-sensitive primary rows

Target	Comparison	Δ mean [95% interval]	Power-sensitive reading
LightFTP†	AFLNet/ChatAFL, `b_abs`	+0.7 [+0.0, +1.7]	$q = .093, \delta = +0.30$; saturated, not coverage-breadth evidence.
bftpd	ChatAFL, `b_abs`	+9.2 [−0.1, +19.8]	$q = .169, \delta = +0.39$; medium effect, interval touches zero.
ProFTPD	ChatAFL, `b_abs`	+41.5 [−53.6, +145.6]	$q = .623, \delta = +0.14$; wide interval and weak rank effect.
Kamailio	ChatAFL, `b_abs`	+88.0 [−9.7, +192.7]	$q = .365, \delta = +0.26$; positive mean, inconclusive.
Kamailio	ChatAFL, `edges`	+11.9 [+2.1, +21.8]	$q = .063, \delta = +0.53$; large effect, misses BH.
Forked-daapd	AFLNet, `edges`	+1.0 [−0.2, +2.2]	$q = .179, \delta = +0.37$; medium effect, interval touches zero.
Forked-daapd	ChatAFL, `edges`	+1.6 [+0.1, +3.1]	$q = .131, \delta = +0.42$; medium effect, not corrected-significant.
Lighttpd1	ChatAFL, `edges`	+2.8 [−4.0, +9.7]	$q = .613, \delta = +0.15$; small, high-variance edge difference.

Table 18. Branch-coverage accumulation metrics

Target	0.95× best final `b_abs`	T_{95} branch coverage (min)			LoopFuzz branch-AUC ratio	
		AFLNet	ChatAFL	LoopFuzz	vs. AFLNet	vs. ChatAFL
LightFTP†	68.1	53	55	54	1.008	1.012
bftpd	475.3	353	178	60	1.034	1.024
ProFTPD	5079.6	NR	332	248	1.079	1.011
Pure-FTPd	1164.1	NR	1304	652	1.299	1.052
Exim	3693.7	785	1003	338	1.030	1.039
Live555	2928.1	NR	NR	57	1.074	1.063
Kamailio	9874.0	577	330	293	1.025	1.006
Forked-daapd	2184.5	455	763	NR	0.976	0.996
Lighttpd1	2037.7	NR	NR	270	1.115	1.083

Table 19. Endpoint robustness diagnostics

Target	Final `b_abs` CV			LoopFuzz `edges` CV	Boundary reading
	AFLNet	ChatAFL	LoopFuzz		
LightFTP	0.000	0.000	0.022	0.048	Saturated near 71; weak branch-breadth evidence.
bftpd	0.028	0.031	0.015	0.076	Lowest LoopFuzz branch CV; faster accumulation.
ProFTPD	0.020	0.019	0.025	0.069	Higher mean; slightly higher branch dispersion.
Pure-FTPd	0.072	0.026	0.021	0.087	Lowest LoopFuzz branch CV.
Exim	0.011	0.013	0.011	0.061	AFLNet-level branch CV with higher mean.
Live555	0.008	0.004	0.007	0.054	Near-AFLNet branch CV with higher mean.
Kamailio	0.012	0.010	0.014	0.075	Modest gains; slightly higher branch CV.
Forked-daapd	0.033	0.022	0.104	0.069	Negative case: lower mean, larger dispersion.
Lighttpd1	0.010	0.019	0.052	0.332	Branch mean improves; edge dispersion high.

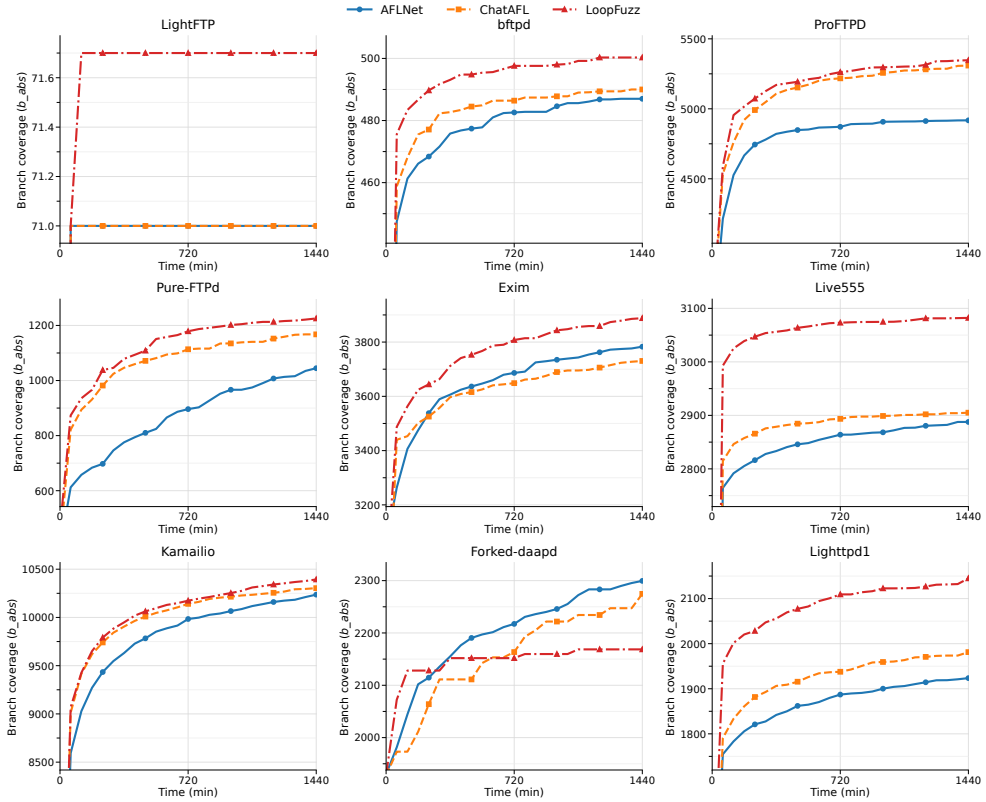


Figure 3. Post-initial ($t > 0$) y-zoomed branch-coverage trajectories for the nine primary-result targets with LightFTP as a saturated artifact-boundary case

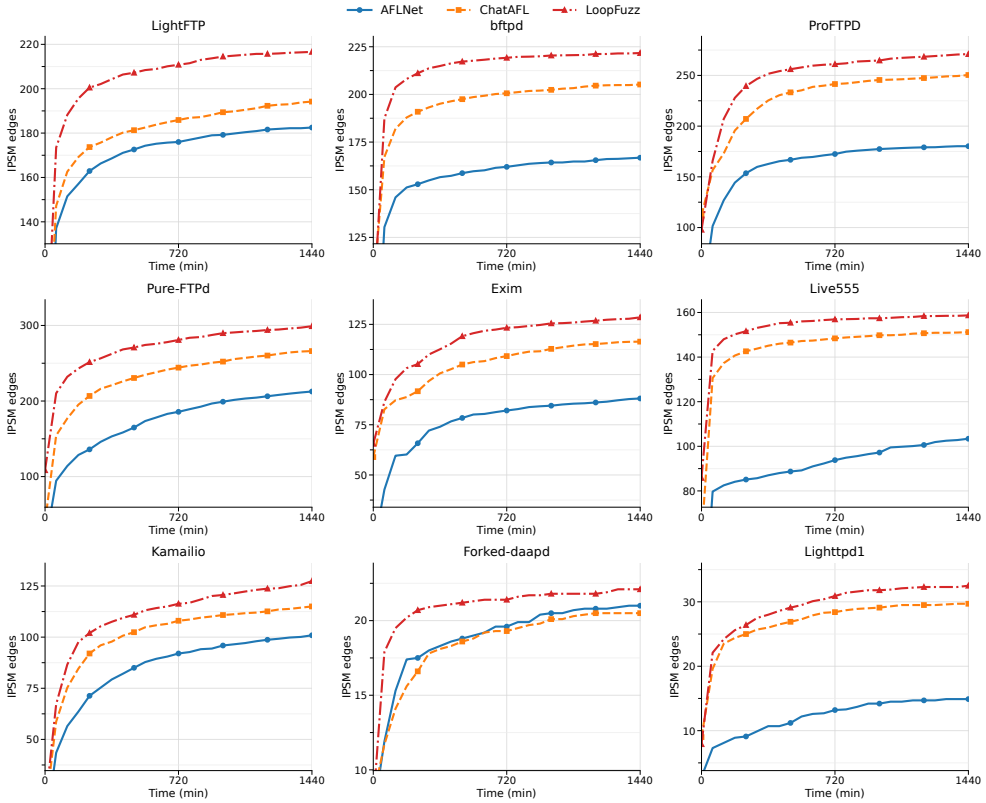


Figure 4. Post-initial ($t > 0$) y-zoomed IPSM-edge exploration trajectories for the nine primary-result targets under the selected-run set from Table 15

Table 20. Auxiliary NSFuzz branch diagnostics

Target	NSFuzz artifact branch count	Target	NSFuzz artifact branch count
LightFTP	414.3±9.8	Exim	4191.5±62.0
bftpd	477.6±3.9	Live555	3021.5±42.1
ProFTPD	5335.0±86.9	Kamailio	10635.3±123.8
Pure-FTPd	1332.2±8.8	Forked-daapd	2722.8±44.6

Table 21. Auxiliary state-exploration diagnostics

Target	AFLNet `edges`	ChatAFL `edges`	LoopFuzz `edges`	NSFuzz artifact edge count
LightFTP	182.8±10.2	194.2±10.2	**216.6±10.4**	11.0±0.0
bftpd	166.9±9.4	205.2±9.7	**221.7±16.9**	48.7±4.8
ProFTPD	180.5±23.1	250.5±9.1	**271.7±18.8**	186.0±11.5
Pure-FTPd	213.9±11.9	266.2±14.3	**299.6±26.2**	17.6±0.5
Exim	88.3±5.3	116.4±10.2	**128.5±7.8**	13.9±1.2
Live555	103.9±10.1	151.2±3.9	**158.7±8.5**	49.8±2.0
Kamailio	101.1±7.7	115.7±13.9	**127.6±9.6**	14.3±0.7
Forked-daapd	21.1±1.3	20.5±2.1	**22.1±1.5**	27.0±6.1

Table 22. Mosquitto selected-summary diagnostics

Artifact	Runs	Runtime (min)	Branch metric	Edge metric	Role
AFLNet Mosquitto	10	1479.0±0.0	`b_abs` 1814.3±147.3	`edges` 36.0±4.5	Aux. summary
LoopFuzz Mosquitto	8	1461.4±13.5	`b_abs` 2933.8±134.4	`edges` 132.2±14.9	Aux.; no ChatAFL
MBFuzzer Mosquitto	10	≈ 1480	replay branches 2827.0±58.9	N/A	External replay

The robustness picture is not uniformly favorable. On ProFTPD and Kamailio, LoopFuzz improves mean endpoints but has slightly higher branch CV than both baselines. On Lighttpd1, LoopFuzz improves final `b_abs` but has higher branch CV and a high response-derived edge CV, which means the state signal is variable even when the mean branch endpoint improves. LightFTP is a saturated target: baseline branch CV is zero because the mean endpoint is essentially fixed at 71, so the small LoopFuzz branch gain should not be used as coverage-breadth evidence. Forked-daapd is the strongest negative boundary: LoopFuzz loses in final branch coverage, fails to reach the common T_{95} threshold, and has branch CV 0.104 versus 0.033 for AFLNet and 0.022 for ChatAFL. Together, Lighttpd1 and Forked-daapd show that IPSM edges can either under-explain branch gains or over-suggest state progress; Sections 7.1 and 6.7 analyze these cases directly.

External artifacts are boundary evidence rather than robustness rankings. NSFuzz branch counts live on a different metric surface. The Mosquitto material now gives AFLNet and LoopFuzz rows on the selected-summary surface, but it still lacks the controlled ChatAFL cell needed for the primary three-way matrix and lacks MBFuzzer AFLNet-lineage `b_abs`/`edges`. A same-pipeline external ranking would require rerunning NSFuzz/MBFuzzer under shared build, instrumentation, replay, budget, and metric extraction procedures, or converting LoopFuzz to their metric surfaces.

6.4. Plateau Activation Analysis

Plateau activation differs enough across targets that a single aggregate LLM-intervention story would be misleading. We stratify targets by the mean archived plateau-call count in Table 13. The high-activation stratum contains targets with at least 60 plateau calls per run, which is close to the 64-call runtime budget. The low-activation stratum contains targets with fewer than 32 plateau calls per run; no primary target falls between these thresholds. This grouping is descriptive rather than causal because plateau calls are triggered by target-specific campaign history, not assigned independently of the fuzzing process.

The high-activation group shows the expected heavy use of the plateau path: LightFTP, bftpd, Pure-FTPd, Live555, and Lighttpd1 all run close to the 64-call budget. Their mean endpoint deltas against the controlled ChatAFL baseline are positive for both branch coverage and response-derived IPSM edges. This does not mean that all of their branch gains came from runtime model intervention. LightFTP is saturated for branch coverage, Live555 has 0.0±0.0% sampled `Hyp. pass`, and Lighttpd1 has a positive branch delta but only a small mean edge delta against ChatAFL. The safer reading is that high activation gives the controller many opportunities to apply plateau assistance, but the archive still lacks candidate-level decisions needed to attribute individual gains to admitted model proposals.

The low-activation group rules out a simple dose-response explanation. ProFTPD, Exim, and Kamailio show positive branch and edge deltas despite only about 14–18 plateau calls per run. Forked-daapd is the negative boundary: it has more plateau calls than the other low-activation targets but lower branch coverage than ChatAFL. ProFTPD is also instructive because its total LLM-call count remains high despite low plateau activation; the difference comes from non-plateau LLM paths such as the inherited template/seed-enrichment pipeline and hypothesis or repair calls. For Exim and Kamailio, the combination of low plateau counts, zero sampled `Hyp. pass`, and positive endpoint deltas means that the current archive cannot justify a claim that frequent plateau intervention or successful grammar-hypothesis validation was the dominant source of improvement. The plausible sources are sparse state-aware plateau hints, initial enrichment, frontier scheduling, and ordinary mutation under the modified scheduler, but the artifact does not contain the event joins needed to separate those paths.

6.5. Bug and Crash Triage

Security findings are reported as a separate artifact-derived triage rather than as part of the endpoint ranking above. The

Table 23. Plateau-activation strata

Target	Stratum	Plateau calls	LLM calls	Tok. total (k)	Hyp. checks	Hyp. pass	Δb_abs vs ChatAFL	Δedges vs ChatAFL
LightFTP†	High	64.0	151.1	179.5	1162.1	0.0%	+0.7	+22.4
bftpd	High	64.0	246.6	179.3	891.5	5.4%	+9.2	+16.5
Pure-FTPd	High	64.0	228.2	496.1	1293.0	8.4%	+61.3	+33.4
Live555	High	63.5	301.1	381.9	754.2	0.0%	+177.6	+7.5
Lighttpd1	High	60.7	165.2	182.9	2715.3	0.0%	+169.0	+2.8
High mean	–	63.2	218.4	283.9	1363.2	2.8%	+83.6	+16.5
ProFTPD	Low	14.1	196.7	103.6	436.8	2.7%	+41.5	+21.2
Exim	Low	15.5	105.4	58.6	266.1	0.0%	+162.3	+12.1
Kamailio	Low	17.5	51.3	114.3	3928.4	0.0%	+88.0	+11.9
Forked-daapd	Low	24.4	59.9	161.1	1722.8	0.0%	−143.4	+1.6
Low mean	–	17.9	103.3	109.4	1588.5	0.7%	+37.1	+11.7
Low mean, excluding Forked-daapd	–	15.7	117.8	92.2	1543.8	0.9%	+97.3	+15.1

available triage worksheets analyze the selected LoopFuzz cells. They do not provide the same triage surface for AFLNet, ChatAFL, NSFuzz, or MBFuzzer, so Table 24 should not be interpreted as a cross-fuzzer vulnerability-discovery comparison.

The recoverable unit is a replayable artifact entry. For crashes, the table counts distinct replayable crash seed files after AFLNet-style replayability filtering; this is a seed-level unit, not a uniform stack-hash or root-cause deduplication across all targets. For non-crash findings, the table counts replayable protocol-oracle violation records and groups them by vulnerability type, Common Weakness Enumeration (CWE), and Common Vulnerabilities and Exposures (CVE) reference pattern. CVE identifiers in the source worksheets are treated conservatively as known-bug or same-class references, not as newly assigned CVEs for the current seeds. Table 24 also uses abort-signal (SIGABRT), AddressSanitizer (ASAN), use-after-free (UAF), User Datagram Protocol (UDP), and denial-of-service (DoS) labels as worksheet-level triage tags.

This triage narrows the security-positioning gap for the paper: the benchmark campaigns do reach security-relevant behaviors, including ASAN-detected crashes, DoS hangs, authentication/state-machine violations, path traversal, request smuggling, CRLF injection, and information disclosure patterns. The evidence remains preliminary. The current paper therefore treats Table 24 as artifact evidence for security relevance, while the main quantitative claims remain campaign-control, coverage, and response-derived state-exploration claims.

6.6. Auxiliary LightFTP Branch Surface

LightFTP demonstrates why the auxiliary NSFuzz rows cannot be read as same-metric rankings. It has the largest gap between the primary-lineage `b_abs` values and the auxiliary NSFuzz branch-count artifact, and the mismatch goes in the opposite direction for edge counts. This discrepancy should not be interpreted as a pipeline error in either artifact or as a fair head-to-head ranking; it shows that the two archives expose different metric surfaces. In the primary table, all three AFLNet-lineage systems use the same LightFTP target revision, fuzzing instrumentation, coverage instrumentation, replay procedure, target binary, and reset hook. Under that shared pipeline, the branch-coverage trajectory reaches 71.0 by 60 minutes for AFLNet and ChatAFL, and 71.7 by 120 minutes for LoopFuzz, then remains flat through 1440 minutes.

The NSFuzz artifact for the same nominal target is 414.3±9.8 branch-count units. The accompanying NSFuzz artifact edge count is 11.0±0.0, while AFLNet, ChatAFL, and LoopFuzz report 182.8±10.2, 194.2±10.2, and 216.6±10.4 response-derived IPSM edges. This combination makes LightFTP the strongest example of cross-artifact metric divergence in the selected data, not evidence that either surface is a same-pipeline ranking of the other.

LightFTP is therefore reported in Table 15 for transparency in the same-build AFLNet/ChatAFL/LoopFuzz lineage comparison and in Table 20 for the auxiliary NSFuzz branch-count surface. Its role in the paper is not to support a branch-coverage breadth claim; it is useful mainly for showing cross-artifact metric divergence and for analyzing response-derived IPSM behavior within the AFLNet-lineage systems.

6.7. Failure-Target Analysis: Forked-daapd

Forked-daapd is the clearest primary branch-coverage exception for LoopFuzz and therefore a design-boundary case for the current state signal. In Table 15, LoopFuzz ends at 2171.9±226.0 final `b_abs`, below AFLNet at 2326.3±77.3 and ChatAFL at 2315.3±50.1. The variance is also diagnostic: LoopFuzz's branch-coverage standard deviation is about three times AFLNet's and more than four times ChatAFL's. Table 20 also reports a high NSFuzz artifact branch count of 2722.8±44.6 for this target. The same target separates state exploration from branch coverage: LoopFuzz improves response-derived IPSM exploration while missing branch coverage reached by the AFLNet-lineage baselines.

The endpoint data point to a state-signal mismatch rather than a simple coverage win. Forked-daapd is the only primary target where LoopFuzz loses `b_abs` by mean and corrected significance, yet it still has the highest mean AFLNet-lineage `edges` (22.1±1.5 versus 21.1±1.3 for AFLNet and 20.5±2.1 for ChatAFL), although the edge difference is not significant after correction. Thus, larger response-derived state structure does not translate into deeper branch exploration on this target.

A plausible explanation is that DAAP/DACP exposes a noisy or sparse response signal for the AFLNet-lineage IPSM abstraction. Forked-daapd contains media-library and control-session behavior where requests can change visible responses without exercising branches that dominate `b_abs`. LoopFuzz may therefore promote candidates that look state-progressive but are not branch-productive. The high variance is also compatible with initialization or early-history instability. The current archive supports this explanation only at the endpoint level; it does not prove which candidate or IPSM transition was branch-unproductive.

To make this boundary measurable in a future artifact, we define a *productive IPSM edge* as a response-derived IPSM edge whose discovering execution or admitted candidate is associated with at least one campaign-value signal: new branch coverage, a newly favored queue entry, or a subsequent branch-coverage gain attributable to descendants of that edge within a declared attribution window. The complementary diagnostic

Table 24. Replayable security-artifact triage boundaries

Target	Crash seeds	Logical rec./classes	Replay status	CVE/CWE boundary
LightFTP	0	291 / 7	7/7 logical replayed.	Same-class refs; no new CVE.
bftpd	500 SIGABRT/ASAN	618 / 8	Crash replay; 4/8 logical confirmed.	CVE-2025-11947-related; refs only.
ProFTPD	62 SIGABRT ASAN heap-UAF	1142 / 6	Docker crash replay; logical config missing.	Heap-UAF preliminary; no new CVE.
Pure-FTPd	0	1164 / 7	Logical AFLNet-style replay/script available.	CVE-2024-48208 not reproduced; refs only.
Exim	0	1050 / 6	1 SMTP class confirmed; others partial.	CVE-2023-51766 family; refs partial.
Live555	72 SIGABRT/ASAN	298 / 6	Persistent replay; standalone incomplete.	Same-class Live555 refs; preliminary.
Kamailio	68 SIGABRT/ASAN heap corruption	275 / 6	UDP/Docker crash + logical replay.	Same-class SIP/Kamailio refs; no new CVE.
Forked-daapd	39 ASAN + 107 DoS hangs	53 / 4 + DoS	4 non-DoS classes confirmed; ASAN/restart needed.	OwnTone/HTTP refs; root cause pending.
Lighttpd1	0	123 / 8	5/8 replayed; others config-dependent.	Same-class HTTP refs; no new CVE.
Mosquitto	Pending	Pending	No worksheet.	Follow-up.

is the fraction of state-progressive candidates or edges that are branch-unproductive. Computing this metric requires an edge id, discovering execution or candidate id, admission decision, coverage delta, favored-queue flag, and downstream descendant attribution. The selected Forked-daapd archives preserve final `ipsm.dot`, queue files, retained raw LLM suggestion files, and run-level `fuzzer_stats`; they do not preserve the normalized event rows needed to join these fields.

The ablation row points away from a simple "one switch is too strict" explanation. On Forked-daapd, removing local repair, frontier scheduling, hypothesis validation, or adaptive triggering all lowers `b_abs` relative to the full controller (-104.4, -145.0, -129.9, and -91.8). Some relaxed variants slightly increase `edges`, but not branch coverage. The target may need a different admission signal, such as richer DAAP/DACP response parsing, target-state normalization, productive-edge accounting, or database/session initialization checks. Until candidate-level and edge-productivity logs are available, the state-signal mismatch remains a reasonable explanation rather than a quantified mechanism.

6.8. Ablation: Boundary-Condition Analysis

Table 26 reports the archived policy perturbations as deltas relative to the full-controller rows in the separate ablation archive. Each cell shows Δ `b_abs` / Δ `edges`. The perturbations measure exposed sub-policy sensitivity within the integrated policy; the full-controller column is a local ablation baseline, not the selected primary-table LoopFuzz cell. Table 27 adds exploratory inference for the same perturbations. Deltas and confidence intervals are computed as perturbation minus local full controller, so negative values favor the full controller.

The statistical picture is weaker than a mechanism-necessity claim would require. Across 80 endpoint comparisons (nine primary targets plus auxiliary Mosquitto, four perturbations, and two metrics), no ablation-vs-full comparison survives BH correction at $q < 0.05$; the smallest adjusted value is $q = .329$. Several raw comparisons are suggestive in isolation, but they remain exploratory after correction. For example, disabling hypothesis validation on Exim reduces `b_abs` by -46.5 branches with a bootstrap interval excluding zero and a large Cliff effect ($p = .007$, $q = .329$, $\delta = -.72$). Disabling hypothesis validation on ProFTPD reduces response-derived `edges` ($p = .013$, $q = .329$, $\delta = -.69$), disabling adaptive triggering on Forked-daapd reduces `edges` ($p = .017$, $q = .329$, $\delta = -.63$), and disabling hypothesis validation on auxiliary Mosquitto reduces `b_abs` ($p = .034$, $q = .329$, $\delta = -.60$). These rows identify targets worth follow-up, not corrected evidence that the corresponding mechanism is necessary across the benchmark.

The mean-delta patterns should therefore be read descriptively. Removing frontier scheduling lowers mean `b_abs` on eight of nine primary targets and also on auxiliary Mosquitto, but the corrected tests do not establish a broad frontier-control effect. Removing local repair raises mean `b_abs` on ProFTPD and Live555 while lowering mean `edges`, yet both branch deltas have wide intervals and weak raw tests; this is at most a hypothesis that repair can over-constrain some target-specific search regions. Removing adaptive triggering raises both mean endpoints on Kamailio, but the intervals include zero and the corrected test does not support a reliable gain from disabling the trigger. On Mosquitto, all four perturbations reduce mean `b_abs` while edge deltas are mixed, reinforcing its role as auxiliary sensitivity evidence rather than a primary benchmark claim. The ablation archive thus supports a cautious conclusion: the full controller is a reasonable integrated policy in this archive, while the exposed sub-controls are target-dependent and not statistically established as individually necessary on every target.

These ablations should also be read together with the low sampled `Hyp. pass` rates in Table 13. The table does not report request-admission pass rates, but it does show that sampled grammar-hypothesis validation rarely succeeds on several targets. Because repair-trigger counts and post-repair gain deltas are not recoverable, the current evidence does not establish the Grammar Hypothesis Loop or local repair as the dominant empirical source of the endpoint improvements. The stronger supported claim is that the integrated policy remains useful despite sparse hypothesis-validation success, with frontier control, plateau gating, admission checks, and repair acting as target-dependent sub-controls rather than uniformly validated independent contributors.

7. Discussion

The paper makes one central design claim. In stateful protocol fuzzing, LLM-generated requests should be governed by runtime evidence before they change the durable queue. ChatAFL already showed that an LLM can generate protocol text [18]. LoopFuzz asks a different question: when should such output be trusted by the campaign? Our design answer is an integrated closed-loop control strategy. It checks structure and parseability before a request trial, then uses the trial response, state movement, and coverage gain to decide whether an LLM suggestion can strengthen the queue. Scheduling-only actions are weaker control hints rather than admitted inputs. The evaluated strategy combines request admission, local repair, and state-aware scheduling as one campaign-control policy; the available archive does not separate the effect of request admission from the other closed-loop controls.

Admission control and scheduling solve different parts of the same problem: one filters low-value suggestions, the other

Table 25. Forked-daapd design-boundary audit

Quantity	Forked-daapd value/status	Interpretation
Final `b_abs`	AFLNet 2326.3±77.3; ChatAFL 2315.3±50.1; LoopFuzz 2171.9±226.0.	Lower LoopFuzz mean and higher variance; primary branch-coverage negative case.
Final response-derived IPSM `edges`	AFLNet 21.1±1.3; ChatAFL 20.5±2.1; LoopFuzz 22.1±1.5.	Highest LoopFuzz mean edge count; no branch-coverage translation.
LoopFuzz plateau calls	24.4±11.4 calls/run.	LLM path active; calls alone do not identify admitted or branch-productive candidates.
Candidate admissions	Not recoverable.	Raw suggestions/queues exist; no candidate id, $P/U/R/G$ decision, or promotion link.
Productive-edge ratio	Not recoverable.	`ipsm.dot` has edges; no links to coverage deltas, favored entries, or descendant gains.
State-progressive but branch-unproductive ratio	Not recoverable.	No candidate-level join for IPSM movement without branch gain.

Table 26. Policy-perturbation endpoint deltas

Target	Full	No local repair	No frontier	No hypothesis	No adaptive trigger
LightFTP	71.6 / 213.9	-0.2 / -13.5	-0.6 / -14.2	-0.5 / -11.0	-0.6 / -1.3
bftpd	501.5 / 221.7	-3.1 / -7.5	-3.0 / -8.4	-5.2 / -5.0	-7.0 / -12.1
ProFTPD	5395.9 / 284.2	+15.8 / -16.3	+30.0 / -23.4	-69.6 / -22.1	-17.0 / -8.9
Pure-FTPd	1240.5 / 299.6	-23.6 / -18.3	-18.5 / -5.3	+4.6 / +0.9	-22.3 / -4.6
Exim	3908.5 / 128.5	-29.0 / -2.8	-39.7 / -2.9	-46.5 / -4.6	-27.7 / -4.2
Live555	3088.7 / 158.7	+1.2 / -1.5	-11.1 / -6.9	-12.2 / -4.4	+5.0 / -3.6
Kamailio	10402.7 / 127.6	-96.7 / +2.5	-121.4 / -5.0	-100.6 / +1.9	+26.9 / +5.2
Forked-daapd	2171.9 / 22.1	-104.4 / -0.9	-145.0 / +0.1	-129.9 / +0.7	-91.8 / -1.7
Lighttpd1	2156.5 / 32.5	-62.4 / -3.3	-15.9 / +1.9	-33.9 / -2.4	-27.1 / +4.1
Mosquitto	2933.8 / 132.2	-172.1 / +3.2	-167.9 / +6.3	-214.8 / -5.2	-132.3 / +4.4

decides where to search. LoopFuzz combines them because filtering and state-directed search are meant to reinforce each other. The archive supports this as an architectural and endpoint-level interpretation, but it does not report candidate-filter counts, predicate failures, or avoided queue pollution.

A stronger semantic-drift discussion would include archived trace case studies joining live state summary, raw LLM candidate, bounded-trial response, $P/U/R/G$ decision, queue outcome, and repair consequence. The current archive preserves raw plateau replies, run-level counters, queue artifacts, response-derived IPSM files, and replayable artifacts, but not that join. Fig. 1 is therefore illustrative, and empirical semantic-drift prevalence remains future candidate-level logging work.

7.1. When IPSM Edges Are Informative

The results clarify the role and the limits of response-derived state counts. A campaign can accumulate superficially different rejection paths without moving toward high-value logic, and a target can also gain branch coverage through paths that do not materially expand the response-derived IPSM. LoopFuzz therefore reads state exploration together with `b_abs` and demotes states that remain unproductive over repeated selections. In this setting, productive states matter more than raw state counts, so `edges` functions as a companion diagnostic for campaign behavior alongside absolute branch coverage.

A true productive-edge metric would count only a response-derived IPSM edge whose discovering execution or admitted candidate is associated with new branch coverage, a newly favored queue entry, or later branch gain from descendants within a declared attribution window. The current archive does not preserve the edge identifier, discovering candidate/execution identifier, admission decision, coverage delta, favored-queue flag, and descendant attribution needed for that metric. Table 28 therefore reports a narrower endpoint-level proxy, Δ`b_abs`/Δ`edges`, using LoopFuzz-minus-ChatAFL mean endpoints from Table 15. This proxy is not a productive-edge ratio; it is a sanity check for whether edge gains and branch gains move in compatible directions.

The informative cases are the state-rich response-visible protocols where response codes and protocol phases tend to co-vary with meaningful session progress. Even there, the endpoint proxy varies by more than an order of magnitude across targets, so edges should be interpreted as a state-exploration surface rather than a normalized unit of coverage productivity. LightFTP is a separate saturated case: edge differences remain visible, but the branch surface is already flat and should not support a coverage-breadth claim.

The two main boundary cases expose opposite failure modes. On Lighttpd1, branch progress occurs inside a shallow HTTP response-state abstraction: LoopFuzz gains +169.0 branches over ChatAFL but only +2.8 response-derived edges. On Forked-daapd, LoopFuzz ends with slightly more primary-lineage `edges` but lower branch coverage, marking a noisy-response DAAP/-DACP boundary rather than a minor exception. Richer response parsing, target-state normalization, productive-edge logging, and database/session initialization checks are promising directions for this target family.

Counterexample-guided local repair is an auditable mechanism, but not a separately quantified driver in this archive. Failed traces can guide prompt-constrained local repair, and changed fields are logged, but multi-field changes warn rather than automatically fail. Low sampled `Hyp. pass` rates, missing repair-event logs, and the non-significant ablation inference in Table 27 mean that repair and trigger effects should be treated as follow-up hypotheses, not corrected evidence of uniform per-module necessity.

The bug/crash triage adds a security-facing view but does not change the causal interpretation of the main experiment. It

Table 27. Exploratory ablation inference

Target	Perturbation	Δ b_abs [95% interval]	p/q	Cliff δ	Δ edges [95% interval]	p/q	Cliff δ
LightFTP	No local repair	−0.2[−1.5, +1.0]	.584/.794	−0.10	−13.5[−24.8, −2.1]	.041/.329	−0.55
LightFTP	No frontier	−0.6[−1.6, +0.0]	.168/.646	−0.20	−14.2[−23.4, −5.8]	.023/.329	−0.61
LightFTP	No hypothesis	−0.5[−1.6, +0.2]	.543/.794	−0.11	−11.0[−20.6, −2.2]	.150/.646	−0.39
LightFTP	No adaptive trigger	−0.6[−1.6, +0.0]	.168/.646	−0.20	−1.3[−12.6, +8.7]	1.000/1.000	−0.01
bftpd	No local repair	−3.1[−10.4, +3.8]	.520/.794	−0.18	−7.5[−21.1, +6.2]	.364/.787	−0.25
bftpd	No frontier	−3.0[−10.5, +4.3]	.472/.788	−0.20	−8.4[−21.2, +4.6]	.364/.787	−0.25
bftpd	No hypothesis	−5.2[−12.4, +1.0]	.471/.788	−0.20	−5.0[−16.9, +6.7]	.449/.788	−0.21
bftpd	No adaptive trigger	−7.0[−14.8, +0.3]	.185/.646	−0.36	−12.1[−23.4, −0.7]	.120/.646	−0.42
ProFTPD	No local repair	+15.8[−82.3, +115.4]	.596/.794	+0.16	−16.3[−30.7, −3.7]	.030/.329	−0.60
ProFTPD	No frontier	+30.0[−108.1, +163.4]	.391/.788	+0.24	−23.4[−53.3, −2.8]	.086/.572	−0.48
ProFTPD	No hypothesis	−69.6[−178.3, +36.1]	.348/.787	−0.27	−22.1[−36.1, −8.0]	.013/.329	−0.69
ProFTPD	No adaptive trigger	−17.0[−123.2, +86.5]	.713/.867	−0.11	−8.9[−25.0, +6.3]	.288/.787	−0.30
Pure-FTPd	No local repair	−23.6[−61.9, +13.4]	.427/.788	−0.22	−18.3[−39.3, +3.1]	.186/.646	−0.36
Pure-FTPd	No frontier	−18.5[−49.4, +8.7]	.570/.794	−0.16	−5.3[−25.6, +16.1]	.570/.794	−0.16
Pure-FTPd	No hypothesis	+4.6[−25.3, +30.0]	.364/.787	+0.25	+0.9[−20.9, +22.1]	.910/.933	+0.04
Pure-FTPd	No adaptive trigger	−22.3[−52.3, +5.9]	.140/.646	−0.40	−4.6[−24.4, +16.4]	.650/.839	−0.13
Exim	No local repair	−29.0[−56.0, +1.0]	.054/.391	−0.52	−2.8[−9.4, +3.3]	.570/.794	−0.16
Exim	No frontier	−39.7[−86.1, +3.8]	.038/.329	−0.56	−2.9[−9.3, +3.4]	.288/.787	−0.29
Exim	No hypothesis	−46.5[−77.6, −13.1]	.007/.329	−0.72	−4.6[−12.5, +2.3]	.544/.794	−0.17
Exim	No adaptive trigger	−27.7[−54.4, +2.0]	.028/.329	−0.59	−4.2[−11.3, +2.8]	.363/.787	−0.25
Live555	No local repair	+1.2[−18.1, +21.3]	.820/.887	+0.07	−1.5[−7.2, +3.5]	.849/.905	+0.06
Live555	No frontier	−11.1[−49.3, +15.9]	.734/.867	+0.10	−6.9[−18.5, +2.0]	.595/.794	−0.15
Live555	No hypothesis	−12.2[−34.3, +11.5]	.290/.787	−0.29	−4.4[−10.6, +1.2]	.171/.646	−0.37
Live555	No adaptive trigger	+5.0[−18.0, +26.6]	.545/.794	+0.17	−3.6[−9.2, +1.2]	.518/.794	−0.18
Kamailio	No local repair	−96.7[−285.1, +85.0]	.427/.788	−0.22	+2.5[−4.5, +9.5]	.761/.867	+0.09
Kamailio	No frontier	−121.4[−308.3, +55.7]	.473/.788	−0.20	−5.0[−16.2, +6.6]	.472/.788	−0.20
Kamailio	No hypothesis	−100.6[−273.1, +58.9]	.571/.794	−0.16	+1.9[−5.8, +10.1]	.909/.933	+0.04
Kamailio	No adaptive trigger	+26.9[−177.0, +219.4]	.734/.867	+0.10	+5.2[−5.7, +15.0]	.198/.659	+0.35
Forked-daapd	No local repair	−104.4[−271.3, +49.5]	.186/.646	−0.36	−0.9[−2.3, +0.5]	.298/.787	−0.28
Forked-daapd	No frontier	−145.0[−300.0, −6.0]	.041/.329	−0.55	+0.1[−1.4, +1.8]	.756/.867	−0.09
Forked-daapd	No hypothesis	−129.9[−282.6, +8.4]	.173/.646	−0.37	+0.7[−1.1, +2.9]	.938/.950	+0.03
Forked-daapd	No adaptive trigger	−91.8[−260.1, +61.3]	.212/.677	−0.34	−1.7[−2.8, −0.6]	.017/.329	−0.63
Lighttpd1	No local repair	−62.4[−147.1, +26.4]	.345/.787	−0.26	−3.3[−10.7, +3.6]	.466/.788	−0.20
Lighttpd1	No frontier	−15.9[−101.7, +72.7]	.791/.867	+0.08	+1.9[−5.8, +9.8]	.761/.867	+0.09
Lighttpd1	No hypothesis	−33.9[−114.9, +48.5]	.791/.867	−0.08	−2.4[−9.4, +4.5]	.650/.839	−0.13
Lighttpd1	No adaptive trigger	−27.1[−109.5, +58.6]	.910/.933	−0.04	+4.1[−6.8, +15.8]	.762/.867	+0.09
Mosquitto	No local repair	−172.1[−367.4, +10.8]	.146/.646	−0.42	+3.2[−7.4, +14.4]	.789/.867	+0.09
Mosquitto	No frontier	−167.9[−384.5, +27.9]	.237/.729	−0.35	+6.3[−7.7, +20.8]	.408/.788	+0.25
Mosquitto	No hypothesis	−214.8[−377.7, −46.6]	.034/.329	−0.60	−5.2[−16.3, +7.3]	.306/.787	−0.30
Mosquitto	No adaptive trigger	−132.3[−340.6, +79.7]	.460/.788	−0.23	+4.4[−7.1, +17.1]	.722/.867	+0.11

Table 28. IPSM-edge informativeness boundary classes

Class	Targets	Δb_abs	Δedges	Proxy	Core reading
State-rich response-code	bftpd, ProFTPD, Pure-FTPd, Exim, Live555, Kamailio	+9.2 to +177.6	+7.5 to +33.4	+0.6 to +23.7	Branch/edge aligned; companion signal only.
Saturated branch surface	LightFTP	+0.7	+22.4	+0.03	Edge growth after branch saturation.
Low-state HTTP	Lighttpd1	+169.0	+2.8	+60.4	Branch gain under-explained by edges.
Noisy DAAP/DACP	Forked-daapd	−143.4	+1.6	−89.6	Edge gain overstates useful progress.

is recovered from the LoopFuzz cells rather than from a normalized AFLNet/ChatAFL/LoopFuzz triage matrix. It shows that the benchmark artifacts contain security-relevant replayable crashes and protocol-oracle violations, while the primary contribution remains an integrated campaign-control study rather than a claim of superior vulnerability yield.

8. Threats to Validity

Internal validity: LLM-assisted fuzzing remains sensitive to generation variance, target nondeterminism, and timing. We mitigate this with fixed container settings, one configured CCTQ OpenAI-compatible endpoint, fixed call-class sampling settings, bounded retries/timeouts, and repeated 24-hour runs. The controlled ChatAFL-lineage row replaces the original public `gpt-3.5-turbo` surface with this study's relay-facing `gpt-5.4-mini` surface, so the primary comparison is against the controlled lineage baseline, not the original ChatAFL artifact results. No model-side seed is set, `top_p` and penalties are omitted, no immutable provider snapshot is recorded, the prompt-reuse filter is not memoization, and no recorded-action replay mode exists; model variance and controller variance therefore cannot be fully separated. The archive also lacks global co-scheduling, cgroup/host-resource traces, daemon-reset counts, and per-run pseudorandom-generator seeds. Growth-rate plots are descriptive same-quota wall-clock traces rather than CPU-time-normalized throughput claims, and Table 12 gives only timeout-based LLM-path upper bounds because trial count, latency, and ordinary-mutation-only throughput are not recoverable.

Statistical conclusion validity: The primary family contains 36 endpoint tests across nine targets, two metrics, and

two baselines. Tables 16 and 17 report Mann-Whitney p, BH-adjusted q, Cliff's δ, and bootstrap intervals, but $n = 10$, ties, and selected-cache filtering limit power. Corrected discoveries have $|\delta| \geq 0.57$, while some non-corrected rows have medium or large effects; such rows are inconclusive under current power, not no-effect evidence. Selected-cache completeness does not prove launch completeness because the pre-filter ledger is missing, so survivorship-bias risk remains. Ablation rows, including the auxiliary Mosquitto rows, come from a separate archive, and Table 27 shows that no ablation-vs-full comparison survives BH correction; those rows are exploratory sensitivity diagnostics rather than corrected evidence of complete admission-boundary or per-module necessity.

External validity: The primary benchmark is the nine non-MQTT targets with comparable AFLNet, ChatAFL, and LoopFuzz rows; Mosquitto is auxiliary. Evidence is strongest for textual or semi-textual request-response protocols with response-visible progress and AFLNet/ChatAFL-lineage IPSM feedback. We do not claim unchanged transfer to TLS, DTLS, QUIC, SSH, IPsec, or other opaque binary/encrypted protocols, which would need dedicated parseability, rejection, and state-progress adapters. Nor do we claim superiority over all state-aware fuzzers: same-format StateAFL, Bleem, Logos, NSFuzz, and MBFuzzer reruns are unavailable.

Construct validity: Final `b_abs` and `edges` are proxies, not complete measures of semantic progress or execution efficiency. AFLNet-lineage `b_abs` rows share target and replay pipelines, while NSFuzz branch counts remain artifact counts; LightFTP's 414.3±9.8 NSFuzz artifact illustrates the distinction. Runtime admission means structured checks, bounded trials, response acceptability, state reachability, gain signals, and local repair, not formal verification. Because there is no *LoopFuzz-no-admission* arm and no normalized admission log, we cannot reconstruct proposal denominators, $P/U/R/G$ failure counts, admitted counts, admission ratios, trial latencies, per-candidate gains, queue-pollution avoidance, repair effects, productive-edge ratios, or branch-unproductive state-progress ratios. Semantic drift, state-signal mismatch, and repair effectiveness are therefore operational explanations supported by endpoint patterns, not quantified prevalence or mechanism-effect summaries. Table 24 adds preliminary security triage, but not a normalized vulnerability-yield comparison.

Reproducibility validity: The controlled ChatAFL-lineage baseline and LoopFuzz share the configured CCTQ-compatible `gpt-5.4-mini` endpoint, declared archives, and deterministic selection rule, but this is not the original ChatAFL model provenance. The artifact documents container isolation, quotas, endpoints, timeout handling, recovery logic, adapter rules, and model-call settings. It does not include an immutable provider snapshot, public credentials, `top_p`/penalty/`seed` fields, normalized per-call prompt/reply/action/latency/token ledgers, candidate-level admission rows, repair-event rows, full launch metadata, or provider-free recorded-action replay. Tables 8, 9, and 7 report the recoverable inventory and gaps. Mosquitto remains auxiliary: the selected summary has 10 AFLNet rows and eight LoopFuzz rows, but lacks a controlled ChatAFL cell and MBFuzzer AFLNet-lineage `b_abs`/`edges`. A complete package should add the missing model, candidate, overhead, launch, resource, seed, and replay records.

9. Conclusion

The main lesson is about control boundaries, not raw generation ability. For textual or semi-textual request-response protocols, LLM output is most useful when it remains a bounded proposal until live execution evidence supports it. LoopFuzz therefore places generation behind structure checks, bounded trials, response acceptability, state reachability, and campaign-gain tests, while treating scheduling-only outputs as weaker control hints. This lesson does not automatically transfer to opaque binary, encrypted, or tightly formatted protocols; those settings first need adapters that expose parseability, rejection behavior, and state progress to the controller.

A second lesson concerns response-derived IPSM feedback. IPSM edges are useful when response structure co-varies with meaningful protocol progress: they give the scheduler and plateau monitor a state signal beyond branch coverage. They can mislead when the branch surface is already saturated, when the protocol has a shallow response-state abstraction, or when noisy responses create state growth that is not branch-productive. For that reason, response-derived `edges` should be read as a companion exploration signal rather than as a calibrated coverage surrogate. Future artifacts should report productive-edge metrics that join each discovered state edge to coverage deltas, favored queue entries, admission decisions, and descendant gains.

The third lesson is methodological. The present archive supports LoopFuzz as an integrated closed-loop policy, but it does not yet close the mechanism ledger. A decisive follow-up should add a *LoopFuzz-no-admission* arm, normalized candidate-level $P/U/R/G$ logs, repair-event records, bounded-trial overhead accounting, resource traces, and provider-free replay of recorded LLM actions. External competitiveness should likewise be tested only through same-pipeline reruns of NSFuzz, MBFuzzer, and other state-aware systems under shared builds, instrumentation, replay procedures, budgets, and metric extraction. These additions would turn the current endpoint-level design evidence into a sharper causal and reproducibility claim.

CRediT authorship contribution statement

Author contribution roles will be confirmed by all authors before submission and transferred into the submission system or final manuscript.

Declaration of competing interest

The competing-interest declaration will be confirmed by all authors before submission.

Funding

Funding information will be confirmed by all authors before submission.

Data availability

Data and code availability statements will be finalized before submission. The submission package will include an anonymized GitHub artifact exposed to reviewers through `anonymous.4open.science`, together with a supplementary archive uploaded through the submission system. The planned package contains the core code, run scripts, target-version pins, container/build configuration material, protocol and fuzzer configuration material, statistical and table-generation scripts, prompt templates, raw selected and auxiliary summary tables, and representative logs needed to audit the tables in this paper. Credentials, local paths, and machine-specific identifiers will be removed from the review package. At the time of this draft, no public DOI or permanent repository URL has been assigned.

Declaration of Generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI Codex for language editing, LaTeX formatting, and submission-compliance checking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] V.-T. Pham, M. B"ohme, A. Roychoudhury, AFLNet: A greybox fuzzer for network protocols, in: 2020 IEEE 13th International Conference on Software Testing, Verification and Validation (ICST), 2020, pp. 460–465. doi:10.1109/ICST46399.2020.00062.

[2] R. Natella, StateAFL: Greybox fuzzing for stateful network servers, Empirical Software Engineering 27 (2022). doi:10.1007/s10664-022-10233-3, art. no. 191.

[3] J. Ba, M. B"ohme, Z. Mirzamomen, A. Roychoudhury, Stateful greybox fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3255–3272.

[4] F. Hu, J. Ji, H. Shu, Z. Li, T. Liu, C. Zhang, Formatted stateful greybox fuzzing of TLS server, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), 2024, pp. 151–160. doi:10.1109/icst60714.2024.00022.

[5] Q. Zhang, X. Bai, X. Li, H. Duan, Q. Li, Z. Li, Resolverfuzz: Automated discovery of DNS resolver vulnerabilities with query-response fuzzing, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4729–4746.

[6] X. Song, J. Wu, Y. Zeng, H. Pan, C. Zuo, Q. Zhao, S. Guo, MBFuzzer: A multi-party protocol fuzzer for MQTT brokers, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 6179–6197.

[7] Z. Luo, J. Yu, F. Zuo, J. Liu, Y. Jiang, T. Chen, A. Roychoudhury, J. Sun, Bleem: Packet sequence oriented fuzzing for protocol implementations, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 4481–4498.

[8] F. Wu, Z. Luo, Y. Zhao, Q. Du, J. Yu, R. Peng, H. Shi, Y. Jiang, Logos: Log guided fuzzing for protocol implementations, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024, pp. 1720–1732. doi:10.1145/3650212.3680394.

[9] N. Redini, A. Continella, D. Das, G. D. Pasquale, N. Spahn, A. Machiry, A. Bianchi, C. Kruegel, G. Vigna, DIANE: Identifying fuzzing triggers in apps to generate under-constrained inputs for iot devices, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 484–500. doi:10.1109/SP40001.2021.00066.

[10] Y. Dong, S. M. M. Rashid, A. Ranjbar, T. Wu, S. R. Hussain, M. S. Mahmud, A. A. Ishtiaq, K. Tu, T. Yang, CoreCrisis: Threat-guided and context-aware iterative learning and fuzzing of 5g core networks, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 5287–5306.

[11] M. B"ohme, V.-T. Pham, M.-D. Nguyen, A. Roychoudhury, Directed greybox fuzzing, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2329–2344. doi:10.1145/3133956.3134020.

[12] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, A. L. Hosking, Seed selection for successful fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 230–243. doi:10.1145/3460319.3464829.

[13] M. Wu, L. Jiang, J. Xiang, Y. Huang, H. Cui, L. Zhang, Y. Zhang, One fuzzing strategy to rule them all, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1634–1645. doi:10.1145/3510003.3510174.

[14] X. Zhu, M. B"ohme, Regression greybox fuzzing, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 2169–2182. doi:10.1145/3460120.3484596.

[15] J. Ba, G. J. Duck, A. Roychoudhury, Efficient greybox fuzzing to detect memory errors, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12. doi:10.1145/3551349.3561161.

[16] R. Natella, V.-T. Pham, ProFuzzBench: A benchmark for stateful protocol fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 662–665. doi:10.1145/3460319.3469077.

[17] A. Andronidis, C. Cadar, SnapFuzz: high-throughput fuzzing of network applications, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 340–351. doi:10.1145/3533767.3534376.

[18] R. Meng, M. Mirchev, M. B"ohme, A. Roychoudhury, Large language model guided protocol fuzzing, in: 31st Annual Network and Distributed System Security Symposium (NDSS 2024), 2024, pp. 1–17. doi:10.14722/ndss.2024.24556.

[19] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, L. Zhang, Fuzz4All: Universal fuzzing with large language models, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13. doi:10.1145/3597503.3639121.

[20] J. Wang, L. Yu, X. Luo, LLMIF: Augmented large language model for fuzzing IoT devices, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 881–896. doi:10.1109/sp54263.2024.00211.

[21] X. Ma, L. Luo, Q. Zeng, From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter IoT devices, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4783–4800.

[22] D. Wang, Y. Li, Z. Zhang, K. Chen, CarpetFuzz: Automatic program option constraint extraction from documentation for fuzzing, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 1919–1936.

[23] Y. Lyu, Y. Xie, P. Chen, H. Chen, Prompt fuzzing for fuzz driver generation, in: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 3793–3807. doi:10.1145/3658644.3670396.

[24] Y. Deng, C. S. Xia, H. Peng, C. Yang, L. Zhang, Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 423–435. doi:10.1145/3597926.3598067.

[25] Y. Deng, C. S. Xia, C. Yang, S. Zhang, S. Yang, L. Zhang, Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 70:1–70:13. doi:10.1145/3597503.3623343.

[26] P. Srivastava, M. Payer, Gramatron: effective grammar-aware fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 244–256. doi:10.1145/3460319.3464814.

[27] J. Fu, J. Liang, Z. Wu, M. Wang, Y. Jiang, Griffin: Grammar-

free dbms fuzzing, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12. doi:10.1145/3551349.3560431.

[28] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, Y. Xiang, Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, 2021, pp. 337–350. doi:10.1145/3460120.3484543.

[29] C. Salls, C. Jindal, J. Corina, C. Kruegel, G. Vigna, Token-level fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2795–2809.

[30] A. Bulekov, B. Das, S. Hajnoczi, M. Egele, No grammar, no problem: Towards fuzzing the Linux kernel without system-call descriptions, in: 30th Annual Network and Distributed System Security Symposium (NDSS 2023), 2023, pp. 1–16. doi:10.14722/ndss.2023.24688.

[31] A. Fioraldi, D. C. D'Elia, D. Balzarotti, The use of likely invariants as feedback for fuzzers, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2829–2846.

[32] C. Myung, G. Lee, B. Lee, MundoFuzz: Hypervisor fuzzing with statistical coverage testing and grammar inference, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 1257–1274.

[33] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, C. Zhang, NSFuzz: Towards efficient and state-aware network service fuzzing, ACM Transactions on Software Engineering and Methodology 32 (2023) 160:1–160:26. doi:10.1145/3580598.

[34] Y. Zhang, D. Fang, P. Liu, L. Xi, X. Lu, X. Chen, S. Si, L. Sun, MSGFuzzer: Message sequence guided industrial robot protocol fuzzing, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), 2024, pp. 140–150. doi:10.1109/icst60714.2024.00021.

[35] K. Kim, T. Kim, E. Warraich, B. Lee, K. R. B. Butler, A. Bianchi, D. J. Tian, FuzzUSB: Hybrid stateful fuzzing of USB gadget stacks, in: 2022 IEEE Symposium on Security and Privacy (SP), 2022, pp. 2212–2229. doi:10.1109/sp46214.2022.9833593.

[36] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, F. Taquist, DTLS-Fuzzer: A DTLS protocol state fuzzer, in: 2022 IEEE Conference on Software Testing, Verification and Validation (ICST), 2022, pp. 456–458. doi:10.1109/icst53961.2022.00051.

[37] K. Sagonas, T. Typaldos, EDHOC-Fuzzer: An EDHOC protocol state fuzzer, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 1495–1498. doi:10.1145/3597926.3604922.

[38] Q. Zhang, Y. Chu, Y. Shen, J. Liu, H. Shi, Y. Jiang, W. Chang, Tron: Fuzzing Linux network stack via protocol-system call payload synthesis, in: 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025, pp. 3227–3238. doi:10.1109/ase63991.2025.00266.

[39] H. Kim, M. O. Ozmen, A. Bianchi, Z. B. Celik, D. Xu, PGFUZZ: Policy-guided fuzzing for robotic vehicles, in: 28th Annual Network and Distributed System Security Symposium (NDSS 2021), 2021, pp. 1–18. doi:10.14722/ndss.2021.24096.

[40] Y. Aafer, W. You, Y. Sun, Y. Shi, X. Zhang, H. Yin, Android smarttvs vulnerability discovery via log-guided fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2759–2776.

[41] T. Scharnowski, N. Bars, M. Schloegel, E. Gustafson, M. Muench, G. Vigna, C. Kruegel, T. Holz, A. Abbasi, Fuzzware: Using precise MMIO modeling for effective firmware fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 1239–1256.

[42] B. Zhao, Z. Li, S. Qin, Z. Ma, M. Yuan, W. Zhu, Z. Tian, C. Zhang, StateFuzz: System call-based state-aware linux driver fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3273–3289.

[43] D. Ryu, G. Kim, D. Lee, S. Kim, S. Bae, J. Rhee, T. Kim, Differential fuzzing for data distribution service programs with dynamic configuration, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 807–818. doi:10.1145/3691620.3695073.

[44] T. Trippel, K. G. Shin, A. Chernyakhovsky, G. Kelly, D. Rizzo, M. Hicks, Fuzzing hardware like software, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3237–3254.

[45] M. Beckmann, J. Steffan, Coverage-guided fuzzing of embedded systems leveraging hardware tracing, in: Computer Security – ESORICS 2023, 2023, pp. 362–378. doi:10.1007/978-3-031-25460-4_21.

[46] Z. Du, Y. Li, Y. Liu, B. Mao, WindRanger: A directed greybox fuzzer driven by deviation basic blocks, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2440–2451. doi:10.1145/3510003.3510197.

[47] P. Lin, P. Wang, X. Zhou, W. Xie, X. Ren, K. Lu, When control flows deviate: Directed grey-box fuzzing with probabilistic reachability analysis, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering, 2025, pp. 713–725. doi:10.1109/ASE63991.2025.00065.

[48] Y. Chen, R. Zhong, H. Hu, H. Zhang, Y. Yang, D. Wu, W. Lee, One Engine to Fuzz 'em All: Generic language processor testing with semantic validation, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 642–658. doi:10.1109/SP40001.2021.00071.

[49] C. Zhang, X. Lin, Y. Li, Y. Xue, J. Xie, H. Chen, X. Ying, J. Wang, Y. Liu, APICraft: Fuzz driver generation for closed-source sdk libraries, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2811–2828.

[50] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, H. Veith, Counterexample-guided abstraction refinement, in: Computer Aided Verification, volume 1855 of *Lecture Notes in Computer Science*, Springer, 2000, pp. 154–169. doi:10.1007/10722167_15.

[51] J. Jiang, H. Xu, Y. Zhou, RULF: Rust library fuzzing via api dependency graph traversal, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 581–592. doi:10.1109/ASE51524.2021.9678813.

[52] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, S. K. Cha, SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 227–239. doi:10.1109/ASE51524.2021.9678888.

[53] Z. Zhou, Y. Yang, S. Wu, Y. Huang, B. Chen, X. Peng, Magneto: A step-wise approach to exploit vulnerabilities in dependent libraries via llm-empowered directed fuzzing, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1633–1644. doi:10.1145/3691620.3695531.

[54] D. Xie, Y. Li, M. Kim, H. V. Pham, L. Tan, X. Zhang, M. W. Godfrey, DocTer: documentation-guided fuzzing for testing deep learning API functions, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 176–188. doi:10.1145/3533767.3534220.

[55] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, T. Holz, SoK: Prudent evaluation practices for fuzzing, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1974–1993. doi:10.1109/sp54263.2024.00137.

[56] M. B"ohme, L. Szekeres, J. Metzman, On the reliability of coverage-based fuzzer benchmarking, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1621–1633. doi:10.1145/3510003.3510230.

[57] C. Poncelet, K. K. Sagonas, N. Tsiftes, So many fuzzers, so little time: Experience from evaluating fuzzers on the contiking network stack, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13. doi:10.1145/3551349.3556946.